

编号 162120321



南京航空航天大学

本科毕业设计（论文）

题目 基于大语言模型的安全关键软件失效模式影响分析生成方法

学生姓名	吴东东
学号	162120321
学院	计算机科学与技术学院
专业	信息安全
班级	1621203
指导教师	杨志斌教授

二〇二五年六月

南京航空航天大学

本科毕业设计（论文）诚信承诺书

本人郑重声明：所呈交的毕业设计（论文）是本人在导师的指导下独立进行研究所取得的成果。尽我所知，除了文中特别加以标注和致谢的内容外，本设计（论文）不包含任何其他个人或集体已经发表或撰写的成果作品。对本设计（论文）所涉及的研究工作作出贡献的其他个人和集体，均已在文中以明确方式标明。

作者签名： 吴东东

日 期： 2025 年 5 月 16 日

南京航空航天大学

毕业设计（论文）使用授权书

本人完全了解南京航空航天大学有关收集、保留和使用本人所送交的毕业设计（论文）的规定，即：本科生在校攻读学位期间毕业设计（论文）工作的知识产权单位属南京航空航天大学。学校有权保留并向国家有关部门或机构送交毕业设计（论文）的复印件和电子版，允许论文被查阅和借阅，可以公布论文的全部或部分内容，可以采用影印、缩印或扫描等复制手段保存、汇编论文。保密的论文在解密后适用本声明。

论文涉密情况：

☐ 不保密

☐ 保密，保密期（起讫日期： _____ ）

作者签名： 吴东东

导师签名： 杨立斌

日 期： 20 年 月 日

日 期： 20 年 月 日

摘 要

安全关键系统广泛应用于航空、航天、核能等领域，一旦发生故障，可能会造成严重后果。因此这类软件系统在开发过程中必须严格遵守相关的安全标准和规范。架构分析与设计语言（AADL）由于能够对硬件组件以及错误模型进行形式化建模，成为设计安全关键系统的重要工具。失效模式与影响分析（FMEA）是一种经典的安全分析方法，它通过系统地识别可能出现的故障方式并分析其带来的影响，来支持系统的安全设计。但传统的 FMEA 方法过于依赖人工经验，效率不高，难以应对复杂系统。为解决这个问题，本文提出了一种基于大语言模型（Large Language Model）的安全关键软件 FMEA 方法。主要研究工作如下：

（1）针对人工进行失效模式与影响分析耗时耗力的问题，提出基于大语言模型的失效模式与影响分析方法。首先压缩 AADL 信息以适配模型输入，其次通过检索增强生成提供安全领域知识，再结合思维链 Prompt，引导生成标准化、可追溯的 FMEA 条目，最后根据生成的 FMEA 报告扩展 AADL 模型。

（2）对基于大语言模型的失效模式与影响分析方法进行实验分析。进行了三个实验，分别验证了信息压缩的效率、RAG 获取安全关键领域知识的效果以及基于思维链的 FMEA 提示框架相较于直接提示的优势。

（3）设计并开发原型工具 FMEAnalysis。该工具基于 AADL 建模环境 OSATE,用于失效模式及影响分析。为证明该工具的有效性，进行了工业界弹射座椅控制系统的案例分析。实现了从原始 AADL 模型到 FMEA 报告的端到端生成。验证了本文工具方法的有效性。

关键词：大语言模型，AADL，失效模式及影响分析，检索增强生成，提示工程

ABSTRACT

Safety-critical systems are widely used in fields such as aviation, aerospace, and nuclear energy, where failures can lead to serious consequences. Therefore, the development of such software systems must strictly follow relevant safety standards and regulations. The Architecture Analysis and Design Language (AADL), with its ability to formally model both hardware and software components as well as error models (EMV2), has become an important tool in the design of safety-critical systems. Failure Modes and Effects Analysis (FMEA) is a classic safety analysis method that supports system safety design by systematically identifying possible failure modes and analyzing their effects. However, traditional FMEA relies heavily on human expertise, which makes it inefficient and hard to apply to complex systems. To address this issue, this paper proposes a safety-critical software FMEA method based on a Large Language Model (LLM). The main contributions are as follows:

(1) To mitigate the labor-intensive nature of manual FMEA, we propose an LLM-based approach. AADL model information is first compressed to fit LLM input constraints. Then, Retrieval-Augmented Generation (RAG) is used to provide domain-specific safety knowledge. Combined with Chain-of-Thought prompting, the LLM is guided to generate standardized and traceable FMEA entries, which are subsequently used to extend the AADL model.

(2) We conduct experimental evaluations of the proposed method through three experiments, verifying the efficiency of information compression, the effectiveness of RAG in acquiring domain knowledge, and the advantages of the Chain-of-Thought prompting framework over direct prompting.

(3) A prototype tool named FMEAanalysis is designed and developed based on the AADL modeling environment OSATE. To validate its effectiveness, we conduct a case study on an industrial ejection seat control system, demonstrating end-to-end FMEA generation from the original AADL model and confirming the practicality of the proposed method.

Keywords: Large Language Models, AADL, Failure Mode and Effects Analysis (FMEA), Retrieval-Augmented Generation (RAG), Prompt Engineering

目 录

第一章 绪论.....	1
1.1 背景和意义.....	1
1.2 国内外研究现状.....	3
1.2.1 基于模型的安全分析.....	3
1.2.2 大语言模型增强安全分析.....	3
1.3 本文主要工作.....	4
1.4 论文组织结构.....	4
第二章 相关背景知识.....	6
2.1 大语言模型.....	6
2.1.1 微调技术.....	6
2.1.2 检索增强技术.....	7
2.1.3 提示工程.....	7
2.2 AADL.....	9
2.2.1 核心语言.....	9
2.2.2 EVM2 Annex.....	10
2.2.3 OSATE 开源工具.....	11
2.3 失效模式及影响分析.....	12
2.4 本章小节.....	13
第三章 大语言模型驱动的 FMEA 生成方法.....	14
3.1 失效模型影响分析方法总体框架.....	14
3.2 基于层级遍历的 AADL 模型信息压缩方法.....	15
3.2.1 层级遍历压缩方法的设计思路.....	16
3.2.2 信息压缩方法伪代码.....	18
3.3 基于 RAG 的安全关键领域知识增强.....	19
3.3.1 安全关键知识库数据集构建.....	19
3.3.2 RAG 机制的知识检索与注入.....	21
3.4 基于思维链的 FMEA 提示框架.....	23
3.4.1 思维链推理逻辑的设计.....	24
3.4.2 FMEA 提示模板的构造.....	24
3.5 基于 FMEA 报告的 EMV2 附件扩展.....	26
3.5.1 元素映射方法.....	26
3.5.2 基于大语言模型的转换实现.....	28
3.6 本章小结.....	29
第四章 框架效果评估与对比分析.....	30
4.1 AADL 压缩效果验证.....	30
4.1.1 数据集选择.....	30
4.1.2 实验结果.....	31
4.2 RAG 安全关键知识验证.....	31
4.2.1 数据集构造.....	32
4.2.2 实验指标设计.....	32
4.2.3 实验结果.....	33
4.3 提示框架效果验证.....	34
4.3.1 数据集构造及模型选择.....	35

4.3.2 实验指标设计	36
4.3.3 实验结果	37
4.4 本章小节	38
第五章 工具实现与案例分析	39
5.1 工具设计与实现	39
5.1.1 FMEAanalysis 工具设计	39
5.2 案例分析	42
5.2.1 基于 FMEAanalysis 的 FMEA 报告生成	45
5.2.2 基于 FMEA 报告的 EMV2 附件扩展	47
5.3 讨论	47
5.4 本章小节	48
第六章 总结与展望	49
6.1 总结	49
6.2 展望	49
参考文献	50

第一章 绪论

1.1 背景和意义

安全关键软件^[1]（Safety-Critical Software）是指在特定应用领域中，一旦出现故障或者失效情况，就有可能对人的生命安全、财产安全以及环境造成比较严重的影响，所以它的可靠性和安全性的要求，相较于普通软件而言要高出许多。常见的应用场景有航空航天、医疗设备、核电站、智能交通系统等，这些领域的系统对于安全保障有着较高要求，来防止系统故障引发严重后果。

安全关键系统的设计和开发通常需要系统开发过程 and 安全性评估在整个生命周期中相互配合。在不同的安全关键应用领域，也会制定对应的标准流程来进行规范。比如，国际民用航空领域的 ARP4754A^[2] 标准明确了从系统需求获取到功能验证的完整开发流程。ARP4761^[3] 标准给出了贯穿设计、测试以及运维阶段的安全评估办法，融合了故障树分析、失效模式与影响分析等方法，用以识别单点故障与共因风险，航空电子领域的 DO-178B/C 标准规定了机载软件从需求、编码再到验证的整个过程，并且借助软件等级来划分验证的严格程度。国内的军用系统主要依据 GJB2786A《军用软件开发通用要求》、GJB438B《军用软件文档编制规范》以及 GJB/Z142-2004《军用软件安全性分析指南》等标准，对需求分析、六性设计以及危险分析等流程给予规范，这些标准与分析方法共同构建起多层次的质量保障体系，保障复杂系统在设计方面符合规定、验证过程完备、运行过程可靠。

系统安全的定义为：贯彻系统生命周期各个阶段，在系统使用效能以及适应性、时间和费用作为约束的条件下，应用工程和管理的原则、准则和技术，使系统满足可接受的事故风险水平。在 ARP4761 中，系统安全评估首先从功能危害评估（Functional Hazard Assessment, FHA）开始，从功能层面出发对潜在的故障状态进行系统性分析，给出系统设计的安全目标。而后进入系统安全评估的两个核心阶段，既初步系统安全性评估（Preliminary System Safety Assessment, PSSA）和系统安全性评估（System Safety Assessment, SSA）。其中，PSSA 作为一种自上而下的系统检查流程，重点聚焦于系统架构设计阶段，其目的在于保证架构可达成 FHA 里明确的安全目标，PSSA 借助高层次分析为低层次设计奠定基础，此阶段建议采用故障树分析方法，对安全问题给予形式化建模，找出冗余系统中潜在的失效风险。

SSA 是一种自下而上的系统验证流程，用于核查系统设计是否契合安全需求，以及系统实现是否契合预定的安全目标与规范。SSA 借助对底层组件的分析，为上层系统的验证提供支撑，在 SSA 阶段，推荐运用失效模式及影响分析方法，来验证系统的安全性与可靠性。

在传统的方法中，安全关键软件的设计开发与安全分析往往依赖于纸质驱动（Paper-Driven）来进行。而纸质驱动存在着“两张皮”问题，既由于使用纸质的系统设计和纸质的安全分析，而导致两者之间交流不畅通、安全分析效率低。特别的在面对复杂系统的软硬件耦合、交互复杂等问题时，无法进行全面的分析和验证。故此基于模型的安全分析（Model Based Safety Analysis, MBSA）在国内外学术界和工业界都得到了广泛重视，成为了安全关键软件（SCS）安全评估的一种前沿解决方案。

MBSA 作为一种前沿方法，可于同一环境里同时开展系统功能模型与故障模型的构建及仿真工作，解决了传统功能分析和故障分析间数据传递不顺畅的问题，促使系统设计和系统实现无缝衔接，大幅提高安全分析效率并减少人为失误。在 MBSA 中常见的建模语言包括 SysML^[4]（Systems Modeling Language）、AADL^[5]（Architecture Analysis and Design Language）、AltaRica^[6] 和 Simulink 等。AADL 在安全分析中具有独特的优势：它通过 Error-Model Annex 2（EMV2）可以在同一个架构中直接描述故障模式和传播方式，支持故障注入和动态仿真，而且 AADL 支持对软硬件进行统一建模，可在设计阶段验证冗余容错机制与资源调度策略，提升系统的可靠性。相较于传统建模方法侧重于功能实现，AADL 更适合应用于航空电子、轨道交通等对系统安全要求极高的关键领域，便于开展系统级的安全分析。

AADL 已经为故障注入、动态仿真和自动化工件生成奠定了良好的基础，然而在面对复杂系统中大量组件和接口的潜在失效模式时，传统的人工分析方法在效率和准确性上仍有一定局限性。随着机器学习，特别是深度学习技术的发展，大语言模型通过大规模通用数据进行训练，在理解和生成自然语言方面展现出强大能力。鉴于此，本文给出了一种依靠大语言模型的安全关键软件失效模式与影响分析方法，目的在于提高进行 FMEA 分析的效率与准确性。

1.2 国内外研究现状

1.2.1 基于模型的安全分析

近年来，基于模型的安全分析研究在方法论、工具集成和自动化水平等方面都取得了显著进展。Stewart 等人^[7]在 AADL 中引入了安全 Annex 并结合模型进行检查，实现了故障组合枚举及其定量评估，将安全分析深入嵌入 MBSE 环境。Sun 等人^[8]系统化的定义了 MBSA 范式，提出了包括建模范围和自动化水平在内的多维度评价框架，为后续研究提供了统一标准。Sun 与 Fleming^[9]基于 STPA 方法提出 STPA+，借助安全约束校正、过程模型约束与控制器设计校验等技术，弥补了传统 MBSA 输入模型在关键场景覆盖方面的不足。Hu 等人^[10]则针对 AADL/EMV2 模型，提出了基于组件故障树（CFT）的 DFMEA 自动生成方法，利用分层 CFT 自动填充 FMEA 表，大大提升了分析的可追溯性和自动化程度。这些研究推动了基于模型的安全分析从理论研究向工程化落地的发展。

1.2.2 大语言模型增强安全分析

近年来大语言模型技术不断发展，像 GPT 和 DeepSeek 等模型，在语义理解与逻辑推理方面表现出较强能力，越来越多研究尝试把这些大语言模型用于失效模式与影响分析，以此提高分析效率和准确性。El Hassani et al.等人^[11]通过微调大语言模型并结合领域知识，构建了一个数据驱动框架，达成了自动化故障模式识别，提升了 FMEA 的效率。Xia 等人^[12]设计了基于检索增强生成的多代理大语言模型系统，能够动态从 FMEA 知识库提取信息、推理并生成定制化建议，有效提升了 FMEA 的效率和准确性。

而 Razouk 等人^[13]提出了一种利用常识知识图谱技术来提升 FMEA 可理解性的方法，他们通过将 FMEA 文档中的信息转成知识图谱，并利用图嵌入技术完成知识图谱，显著提高了 FMEA 文档的完整性和准确性。而同样的 Bahr 等人^[14]也利用了知识图谱的方法来增强检索增强生成，通过将 FMEA 数据建模为属性知识图谱并引入图查询语言，实现了对 FMEA 数据的精确检索和分析。

这些研究有效表明了大语言模型能显著提升广泛场景下的失效模式识别与风险评估的效率,但是大多数的研究主要聚焦于利用通用大语言模型进行任务处理,针对安全关键软件的专项 FMEA 研究则相对有限。特别的基于模型驱动结合大语言模型进行安全分析的则更少。故此本文提出一种基于大语言模型的安全关键软件失效模式影响分析生成方法，进行模型驱动结合大语言模型进行安全分析。

1.3 本文主要工作

本文提出一种基于大语言模型的安全关键软件失效模型影响分析方法。主要研究工作如下：

（1）针对人工进行失效模式与影响分析耗时耗力的问题，提出基于大语言模型的失效模式与影响分析方法。首先，进行 AADL 信息压缩用于输入大语言模型。其次通过检索增强生成(Retrieval-Augmented Generation, RAG)，为 LLM 提供安全关键领域提供专业领域知识，然后通过思维链（Chain-of-Thought）Prompt 方法,并结合 AADL 组件拓扑关系以及失效案例引导 LLM 生成标准化、可追溯的 FMEA 分析条目。最后根据生成的 FMEA 报告扩展 AADL 模型。

（2）对大语言模型的失效模式与影响分析方法进行实验分析。分别进行了三个实验，验证了信息压缩的效率、RAG 获取安全关键领域知识的效果以及基于思维链的 FMEA 提示框架相较于直接提示的优势。

（3）设计并开发原型工具 FMEAanalysis。该工具基于 AADL 建模环境 OSATE 平台,用于进行失效模式及影响分析。为验证该工具的有效性，使用工业界弹射座椅控制系统的案例进行分析。

1.4 论文组织结构

本论文的组织结构如下：

第一章绪论。阐明课题的研究背景和意义,简要介绍了安全关键软件、安全分析, AADL 语言。并简要概述了研究现状及本课题的主要研究工作。

第二章相关背景知识。介绍相关的背景知识，包括大语言模型的微调技术、检索增强技术、提示工程及其应用。阐明了 AADL 语言核心概念以及 OSATE 工具及其插件生态，最后介绍了失效模式及影响分析方法。

第三章提出基于大语言模型的安全关键软件失效模型影响分析方法。介绍框架总体流程、信息压缩方法、安全关键领域的 RAG 增强、以及基于思维链的 FMEA 提示框架，最后给出基于 FMEA 的 AADL 模型更新方法。解决人工进行失效模式与影响分析耗时耗力的问题。

第四章进行实验验证基于大语言模型的安全关键软件失效模型影响分析方法的高效性。主要进行三个实验，分别验证信息压缩方法、安全关键领域的 RAG 增强、以及基于思维链的 FMEA 提示框架的有效性。

第五章工具实现及案例分析。设计并实现了安全分析工具 FMEAanalysis。介绍工具各个模块功能，并结合工业界案例弹射座椅控制系统进行安全分析，验证本文方法的有效性。

第六章总结全文并展望未来。

第二章 相关背景知识

2.1 大语言模型

2.1.1 微调技术

目前，大语言模型正以极快的速度发展，在语言理解、文本生成、专业问答等多个应用场景中表现优异，展现出强大的能力，如在代码生成、医学问答等多领域取得了“人工通用智能”水平的突破性成果^[15]。但是，在面对下游特定的行业和专业场景时，这些通用模型往往无法提供足够的专业性和准确性。故此，采用微调（Fine-Tuning）技术^[16]成为提升模型在特定任务中表现的重要手段。

微调是指在预训练模型的基础之上，借助特定行业所拥有的数据来对模型展开的训练，以此使得模型可适应特定任务，减少大语言模型在该领域出现错误或者“幻觉”现象的概率，经过微调之后，模型可以学习该领域的专业术语、语言风格以及任务特点，最终在专业领域内给出更为准确且可靠的结果。

随着大语言模型数据规模的持续增大，全面调整所有参数的微调方法在计算资源以及存储资源方面面临着较大的挑战。基于此，近些年来研究人员提出了多种参数高效微调技术，凭借仅仅调整一小部分参数或者增加额外的模块，保证在有限资源下模型微调的效果。常见的高效微调技术包括：

LoRA^[17]是一种轻量化的微调方法，它借助在 Transformer 的每一层插入可进行训练的低秩矩阵，达成对大型预训练模型的高效调整，此方法将原有模型权重给予固定，仅仅对新添加的低秩矩阵展开训练，极大程度地减少了需要训练的参数数量。

BitFit(Bias-Only Fine-tuning)方法^[18]仅更新模型里的偏置参数，其余参数全部处于冻结状态。虽说偏置参数在模型中所占比例较小，不过研究显示，在规模较小到中等的数据集上，BitFit 的表现与全参数微调相近，有时甚至更佳。

Prefix Tuning 方法^[19]是在输入序列之前增添一组可以训练的前缀向量，使得预训练的语言模型在不改动原有参数的情形下，也可适应特定任务。这些前缀向量作为额外的上下文信息，帮助模型生成契合任务要求的结果。

Prompt Tuning 方法^[20]是一种经过简化的微调办法，凭借学习一段可训练的软提示，直接与输入的嵌入进行拼接，适用于少样本场景，该方法并不修改模型参数，而是凭借优化输入提示，提高模型在特定任务上的表现。

Adapter Tuning 方法^[21] 在每个 Transformer 层中添加轻量级的适配器模块，只对这些模块以及层归一化参数进行训练，其他部分维持冻结状态，该方法在多个任务上的表现与全参数微调相近，较大降低了训练和存储成本。

2.1.2 检索增强技术

检索增强生成（Retrieval-Augmented Generation, RAG）^[22] 是一种结合信息检索和文本生成的自然语言处理技术。在面对需要专业领域背景知识的任务时，该技术可依靠实时检索外部知识库，来帮助大语言模型提高回答的准确度以及上下文相关性。具体而言，就是把外部知识检索和大语言模型的生成能力相结合，借助动态引入如技术文档、故障案例库等领域知识库，以此提升模型输出的准确性与可解释性。

RAG 的核心机制包括以下几个步骤：

（1）外部知识库构建：首先要从如 API、数据库或者文档存储库等各类来源，去收集和目标任务相关的外部数据，而这些数据是处于 LLM 原始训练数据范围之外的。

（2）数据预处理与向量化：对收集而来的外部数据开展预处理操作，借助嵌入模型将其转化为数值表示既向量，这些向量会被存放于向量数据库之中，构建可供大语言模型访问的知识库。

（3）用户提出查询：系统会把该查询转化成向量表示，然后在向量数据库里查找与之最为相关的文档，比如在医疗健康领域，患者可向聊天机器人输入症状，像“头痛和发烧”等，之后系统会检索相关的医学文献以及患者历史记录，生成个性化的建议。

（4）增强提示构建：检索到的相关文档会作为外部知识，与原始查询一起整合，形成新的输入提示，用来引导生成模型给出更准确、更丰富的信息。

（5）文本生成与输出：生成模型会依据整合后的提示来生成最终的回答，凭借这种方式，模型可动态地去访问以及利用最新的、特定领域方面的知识，而不需要频繁地对整个模型重新进行训练。

（6）知识库更新与维护：要保证 RAG 系统检索信息的准确性和相关性，需要通过自动流程或定期批处理来更新外部数据库。

2.1.3 提示工程

提示工程是通过设计和优化输入提示，引导大语言模型生成更高质量、可控的输出。它本质上是构建一个“人机协作”的语义接口，把用户的意图准确转化为模型能够理解的指令。通过精心设计提示，用户可以给模型提供明确的任务指令、必要的上下文信息以及

示例，帮助模型更好地理解意图，生成更高质量的回答。这种方式在不改变模型参数的情形下，提升了模型在特定任务里的表现，在问答、文本生成以及推理等应用方面呈现出了出色的效果。

(1)提示工程的基本概念

在大语言模型中，用户通常通过输入一段文本提示与模型互动。这个提示不仅包含自然语言的指令，还可能包括任务背景、上下文信息和具体示例。模型会根据这些提示生成相应的输出。提示工程的关键在于优化输入文本的结构和内容，最大限度地发挥模型的推理和生成能力。其目标是让模型更好地理解任务，从而产生更准确、更高质量的结果。提示工程主要包括以下几个方面内容：

任务定义：既明确提示的目标，像文本生成、问答、翻译、分类等。

输入格式设计：依照具体任务需求构建提示内容，内容可能包含明确的指令、示例以及相关约束条件。

上下文管理：为模型提供必要的背景信息，帮助它更准确地理解并执行任务。

优化与迭代：根据模型的输出表现，不断调整提示内容，实现逐步改进。

(2)提示工程的关键技术

零样本学习^[23]：通过构建通用型提示语句，在不依赖任何训练实例，也无需补充额外语境信息的情况下，直接向模型下达指令或提出问题，用于快速获取模型已有的知识。

少样本学习^[24]：少样本学习则通过为大语言模型提供少量示例，帮助模型更好地理解任务要求。这两种方法在提示工程中被广泛使用，以减少对大量标注数据的依赖。

逆向提示^[25]：通过设想最终想要的输出，反过来去设计最合适的输入提示。这种方法有助于理解模型的表现，并改进提示的设计。

元学习与自适应提示^[26]：元学习可使模型借助少量示例迅速适应新任务，自适应提示会依据任务的变化动态地调整提示内容，以此来提高准确率以及效率。

思维链提示^[27]：这是一种提升模型推理能力的技术，通过分步骤提示，指导模型一步步完成推理。比如在数学题中引导模型逐步解题。

(3)提示工程的应用领域

提示工程在许多自然语言处理任务中都有广泛应用，像是自然语言生成、问答系统与信息抽取、代码生成与调试、情感分析与文本分类以及跨语言任务等，在自然语言生成方面，对提示加以优化，可让模型生成风格统一且符合语境的文本，比如新闻、广告文案、小说等类型的文本，在问答系统与信息抽取里，依靠设计有效的提示，可帮助模型准确理

解问题，从数据中提取或者生成所需信息。在代码生成与调试领域，提示工程可用于编写代码、查找错误或者提供编程建议，提高开发效率，进行情感分析与文本分类时，依靠清晰的说明以及示例提示，能提升模型在分类任务中的表现，而跨语言任务如机器翻译或跨语言信息检索，借助提示设计可提升模型在不同语言环境下的表现能力。

2.2 AADL

2.2.1 核心语言

AADL 即体系结构分析与设计语言，是美国汽车工程师协会 SAE 所制定的建模标准，其主要作用是辅助嵌入式实时系统的设计与分析。它是在 MetaH 和 UML 等建模语言的基础上发展而来，运用组件化方式进行建模，并结合形式化方法描述系统架构。通过 AADL，可以清楚地描述系统中各个组件的结构、行为以及它们之间的交互情况，还可将软件功能对应至具体的硬件平台。AADL 不仅能进行功能建模，还支持对系统的非功能属性如性能、调度能力、可靠性等展开建模与分析，如此便能在系统设计的早期阶段验证一些关键特性，提升系统设计的可行性与安全性^[28]。

在 AADL 中，系统被构建成一种层次结构，是由多个软件以及硬件组件组合而成的，该语言自身有一套预先设定好的组件类型，其中包含软件组件、运行平台相关的组件以及复合组件。这些组件类型对应的是嵌入式系统设计中的实际构成部分，方便架构师从工程角度对系统进行建模。

每个组件的基本特性是由其“组件类型”来进行定义的，这种组件类型会对组件的接口给予描述，其中涉及了可借助通信端口和其他组件展开交互的功能，一个组件类型可拥有多个实现，这些实现会具体说明其内部结构，比如所包含的子组件以及连接方式等，组件类型或者实现还可借助“扩展”机制去继承其他组件，以此对其属性加以细化，或者增添新的子组件以及功能特性。

在建模时，组件需要被实例化为其他组件的子组件，组成系统的层次结构。一个完整的 AADL 模型必须定义一个顶层的系统组件，作为整个架构的根节点。这个顶层系统包括处理器、进程、总线、设备、抽象单元或内存等组件，从而构建出一个可以实例化的完整系统架构，如图 2.1 所示。

此外，AADL 不仅支持结构建模，还支持添加执行时间、内存占用和调度限制等非功能属性，同时也能描述系统的行为。这种能力让 AADL 能够对系统的功能和非功能特性进行建模与分析，适合在系统开发早期阶段开展性能评估、调度验证以及安全分析。

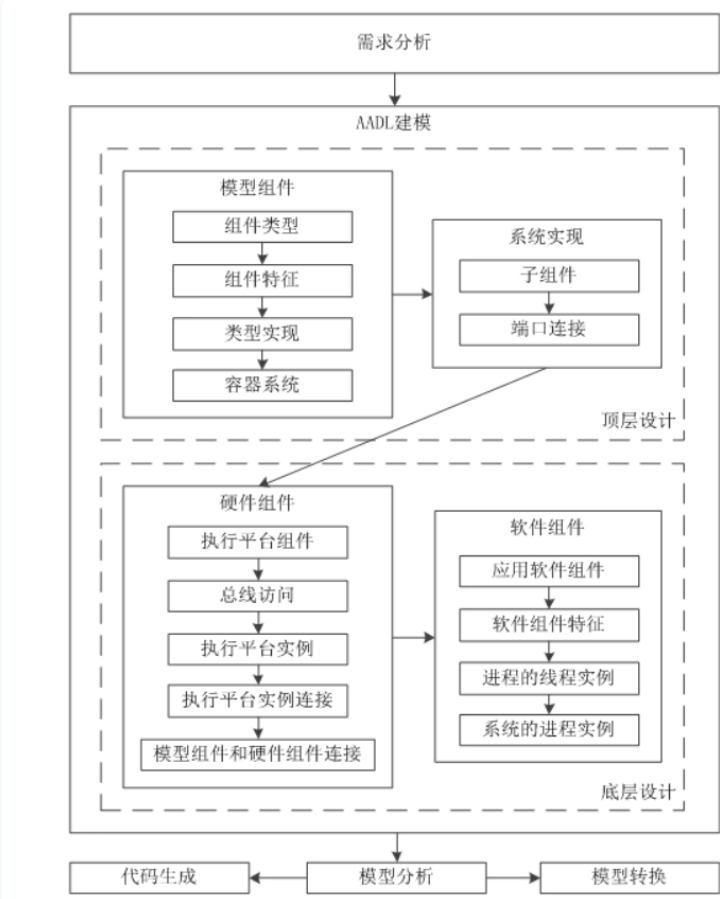


图 2.1 AADL 建模过程

2.2.2 EVM2 Annex

EMV2 附件^[29]（Error Model Annex Version 2, EMV2）是 AADL 的关键扩展之一，为系统架构建模增添系统性安全分析能力，其可在 AADL 模型里纳入和安全有关的建模内容，使用户可清晰描述故障引发的危险、故障传播路径、具体故障模式以及可能产生的影响，还支持对复杂故障组合行为进行建模，为后续自动化安全分析给予有力支撑。

EMV2 引入了“故障传播本体”（Fault Propagation Ontology, FPO）的概念，从三个建模层面支持系统架构的故障建模：

（1）错误传播建模：这一部分用来描述系统组件之间以及它们与外部环境之间的故障传播方式。EMV2 着重关注故障源的建模，以及当故障在组件接口传播时，对其他组件或系统可能产生的影响。

（2）复合故障行为建模：EMV2 可将子组件的故障状态与上层组件的故障状态建立关联，对更为复杂的故障行为进行建模，这种做法可在系统的不同层级间整合并分析故障表现。

（3）错误类型定义：凭借定义不同的错误类型，EMV2 构建了一种描述故障传播含义的基础框架。用户可依据具体系统或行业的需求，定制或扩展错误类型库，以契合特定应用的建模需求。

EMV2 附件主要面向航空航天领域，其设计目的是支持 SAE ARP4761 标准中定义的安全评估流程和方法。通过在系统设计初期引入安全建模，EMV2 有助于工程师发现潜在的风险和故障路径，并提前制定应对方案，实现主动的安全设计。同时，EMV2 还明确了其建模元素与安全评估方法之间的对应关系，比如：错误源、错误汇和错误路径等元素可以直接对应到故障传播与影响分析中。

鉴于 EMV2 附件与故障模式及影响分析在分析理念与建模结构上的高度契合，EMV2 模型可便捷地转换为 FMEA 表达形式，从而在实践中增强安全分析的效率与系统性。而 FMEA 也可扩展 MEV2 附件，从而实现文本化分析结果到模型化安全信息的闭环反馈，提升模型的完整性与一致性。。

2.2.3 OSATE 开源工具

OSATE^[30]（Open Source AADL Tool Environment）是由卡内基梅隆大学软件工程研究所（SEI）开发的开源集成开发环境，主要用于支持基于 AADL（架构分析与设计语言）的系统建模、编译和分析。该工具被广泛应用于航空航天、汽车电子等安全关键系统的设计和验证，能为系统架构建模和安全分析提供有力的技术支持。

OSATE 的核心功能包括以下几个方面：

（1）模型创建与编辑

OSATE 基于 Eclipse 平台，提供了语法智能的文本编辑器和同步更新的图形化建模界面，帮助用户高效地创建、浏览和维护 AADL 模型。它的编辑器可以识别 AADL 语言的语法和语义，提高了建模的准确性和效率。

（2）插件扩展能力

由于 OSATE 构建于 Eclipse 平台之上，其开放的插件架构使其具备高度可扩展性。发者可以根据具体项目需求，自定义和集成各种分析插件。比如，AGREE（Assume Guarantee Reasoning Environment）插件利用模型检查技术，用来验证系统架构是否满足设定的时序约束和安全属性。此外，OSATE 还提供了对 EMV2 Annex 和 Safety Annex 的官方支持插件，满足多层次的安全建模和分析需求。

（3）安全分析方法支持

OSATE 深度集成多种主流的系统安全分析方法，包括但不限于：

功能危险评估：分析系统功能可能带来的潜在危险，明确其对系统安全的影响，帮助制定有效的缓解措施。

故障树分析：通过构建系统故障的逻辑树，找出可能导致系统失效的基本事件，支持对系统整体安全性和可靠性的评估。

故障影响分析及故障模式影响分析：识别系统中的潜在故障模式，评估它们对系统性能和安全的影响，从而提高系统的鲁棒性和容错能力。

得益于其模块化架构设计和丰富的插件生态，OSATE 已逐步成为模型驱动系统安全分析流程中的重要工具平台。

2.3 失效模式及影响分析

失效模式及影响分析^{错误!未找到引用源。}是一种系统化、自下而上的安全分析方法，广泛用于识别、评估和预防潜在失效模式及其对系统性能的影响。FMEA 的主要目标是通过深入分析每种潜在失效模式，系统地找出它们可能对上层系统、子系统或功能单元造成的影响，为设计优化和风险规避提供参考。

FMEA 通常在系统的设计、制造或运行阶段实施，分析过程包括以下关键步骤：

（1）失效模式识别：识别系统或组件中可能发生的失效模式，这些失效可能源于设计缺陷、制造误差、材料疲劳、操作失误或外部干扰等因素。

（2）影响分析：针对每一种失效模式，分析其对系统功能的潜在影响。此过程聚焦于理解故障发生后系统的性能退化、功能丧失或安全隐患。

（3）风险评估维度：FMEA 通过以下三个核心参数对每种失效模式进行量化评估：

- 1) 严重性：失效后果对系统性能、安全性或用户造成的潜在危害程度。
- 2) 发生概率：该失效模式在特定条件下发生的可能性。
- 3) 可检测性：系统在失效发生前或发生时检测出该失效的能力。

为实现风险排序和优先级评估，FMEA 引入了风险优先数（Risk Priority Number, RPN）这一概念。RPN 通常通过三个因素的乘积计算得到，既 $RPN = S \times O \times D$ 。RPN 数值越高，说明该失效模式的综合风险越大，应该优先采取风险控制措施，比如设计改进、优化制造工艺或加强监控等。

FMEA 的优势体现于其方法有系统性、结构化以及可操作性，它可在系统开发的早期阶段识别出潜在问题，降低后期故障修复所产生的成本与风险，这种方法在各类安全关键领域得到了广泛应用，比如在汽车工业当中，FMEA 用于提升零部件以及整车系统的可靠性和耐久性，在航空航天领域，FMEA 属于飞行器系统安全设计与验证的关键构成部分，在制造与医疗设备行业，FMEA 用来优化流程、提高产品质量以及系统鲁棒性。

然而传统的 FMEA 分析主要依靠专家手工去识别以及评估失效模式，在面对数量众多的系统设计文档以及历史故障数据时，手动构建 FMEA 表格效率不高并且容易出现差错，难以契合复杂系统快速迭代情况下对高效安全评估的需求，借助人工智能和自然语言处理技术，自动提取失效模式并生成 FMEA 表格，成为提升安全分析效率与准确性的关键方向。这种自动化方法大大减轻了分析人员的工作压力，又能在大规模系统中达成更全面且更及时的安全评估。

Component Name	Mode	Cause	Effect	Severity	Occurrence	Detection	RPN	Optimization Method	Optimization Effect	Recommended Actions	New RPN
DSP中断寄存器	中断初始化失败	DSP中断寄存器故障	系统响应时间延迟	8	4	3	240	增强寄存器异常检测机制	故障检测覆盖率提高	添加寄存器异常检测机制	120
DSP CPU	系统主频初始化失败	CPU故障	系统时钟紊乱，定时失败	9	3	4	180	引入冗余CPU，增强自检功能	关键组件可靠性显著提升	引入冗余CPU，增强自检功能	90
DSP寄存器模块	寄存器初始化失败	寄存器初始化异常	数据读写错误，系统崩溃	7	6	3	210	增加寄存器初始化冗余校验	初始化成功率提高	增加寄存器初始化冗余校验	105
DSP QSPI存储器	通用IO初始化失败	QSPI存储器异常	通信数据丢失或损坏	7	3	4	140	增加寄存器状态监测与冗余	寄存器状态监测更全面	增加寄存器状态监测与冗余	70
DSP外部中断寄存器	外部中断寄存器初始化失败	外部中断寄存器故障	系统响应时间延迟	8	6	3	240	优化初始化流程与冗余设计	初始化流程更加稳健	优化初始化流程与冗余设计	120
DSP定时器	定时器初始化失败	定时器初始化异常	定时精度偏差，系统异常	7	6	3	210	增加初始化冗余检测机制	初始化成功率提高	增加初始化冗余检测机制	105
DSP事件管理器	事件管理器初始化失败	事件管理器初始化异常	事件事件管理异常	8	3	4	180	改进初始化流程	提升初始化成功率	引入冗余事件管理器	80
854芯片	854芯片寄存器初始化失败	854芯片寄存器异常	电源数据，系统性能下降	9	6	3	210	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	135
Flash存储器	Flash存储器初始化失败	Flash存储器异常	Flash存储器初始化失败	7	3	4	140	优化寄存器初始化算法	提高寄存器初始化成功率	优化寄存器初始化算法	70
串行通信寄存器	串行通信寄存器初始化失败	串行通信寄存器故障	通信数据丢失或损坏	10	3	4	200	增加寄存器冗余检测机制	减少初始化失败导致的通信中断	增加寄存器冗余检测机制	100
AD转换器	硬件异常导致AD上电时间延迟	AD转换器故障	系统启动时间延迟	5	4	6	120	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	60
数据通信系统	数据通信系统异常	数据通信系统故障	数据通信系统异常	6	3	3	180	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	90
开关电源控制系统	开关电源控制异常	开关电源控制故障	系统响应时间延迟	7	4	3	140	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	70
EEPROM模块	EEPROM模块异常	EEPROM模块故障	系统响应时间延迟	6	3	6	108	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	54
EEPROM模块	EEPROM模块异常	EEPROM模块故障	系统响应时间延迟	7	4	6	168	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	84
EEPROM模块	EEPROM模块异常	EEPROM模块故障	系统响应时间延迟	8	3	3	156	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	78
EEPROM模块	EEPROM模块异常	EEPROM模块故障	系统响应时间延迟	9	3	3	171	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	81
软件执行	系统初始化失败	软件执行异常	系统初始化失败	7	4	6	168	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	84
数据通信系统	数据通信系统异常	数据通信系统故障	数据通信系统异常	6	3	3	108	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	54
数据通信系统	数据通信系统异常	数据通信系统故障	数据通信系统异常	7	4	6	168	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	84
数据通信系统	数据通信系统异常	数据通信系统故障	数据通信系统异常	8	3	3	156	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	78
数据通信系统	数据通信系统异常	数据通信系统故障	数据通信系统异常	9	3	3	171	增加寄存器冗余检测机制	降低寄存器异常发生率	增加寄存器冗余检测机制	81

图 2.2FMEA 表样例

2.4 本章小节

本章围绕论文的研究背景，系统介绍了与大语言模型相关的核心技术，包括微调技术、检索增强生成技术和提示工程。接着，详细讲解了 AADL 语言的核心概念、故障模型附件 EMV2 Annex 以及建模工具平台 OSATE，说明了故障模型附件在系统建模与安全分析中的应用。最后，介绍了失效模式及影响分析（FMEA）方法。

第三章 大语言模型驱动的 FMEA 生成方法

本章聚焦于大语言模型的失效模式与影响分析生成方法展开研究工作，提出了面向安全关键系统的大语言模型 FMEA 生成的整体框架，详细介绍了各个功能模块之间的协同流程，接着介绍了针对 AADL 系统实例的层级遍历压缩方法，该方法用于提取 FMEA 所需要的关键信息。然后构建了安全关键领域的专业知识库，并且使用 RAG 的知识提高机制，为模型提供高质量的领域语境支持，随后引入了基于思维链提示的 FMEA 提示工程策略，以此提升大语言模型在 FMEA 任务中的推理能力以及输出的准确性，最后给出基于生成的 FMEA 报告更新 AADL 模型中的 EMV2 附件方法，实现端到端的安全分析。

3.1 失效模型影响分析方法总体框架

失效模式与影响分析在系统安全和可靠性设计中十分重要。它的首要步骤是识别系统、组件或设计中可能出现的失效模式，明确系统在特定条件下可能无法完成预期功能或达到性能要求的情况。FMEA 能够在设计早期发现并减少系统失效风险，为优化设计及预防故障提供关键依据，从而提升产品或系统的安全性和可靠性。传统的 FMEA 依赖领域专家的经验 and 历史故障数据，通过人工逐项检查系统功能模块，系统地梳理潜在风险。但这种人工方式效率较低且带有较强的主观性。为此，提出了一种基于大语言模型的安全关键软件失效模式与影响分析方法

基于大语言模型的安全关键软件失效模式与影响分析方法的总体框架如**错误!未找到引用源。**所示。本方法首先对 AADL 模型进行实例化处理，将其转化为包含系统组件及其交互关系的 JSON 格式数据，形成结构化的系统描述，作为开展 FMEA 分析的基础。随后，引入检索增强生成（RAG）技术，从安全关键领域的知识库中获取相关失效案例与背景知识，为大语言模型提供必要的语境支撑，提升其对专业任务的理解能力。在此基础上，结合基于思维链的提示策略，将结构化数据、领域知识与失效文本共同输入模型，引导其分步骤完成失效识别、影响分析及改进建议的生成。最终，模型以 JSON 格式输出 FMEA 报告，内容涵盖潜在的失效模式、系统影响及建议措施，为后续设计调整提供依据。基于生成结果，可进一步自动更新 AADL 模型中的 EMV2 附件，用于进一步的风险量化和设计改进。该方法将 AADL 建模、RAG 技术与提示工程有机结合，有效优化了传统 FMEA 流程，特别适用于航空航天、汽车电子等对安全性要求极高的领域，在提升分析效率的同时增强了自动化支持能力。

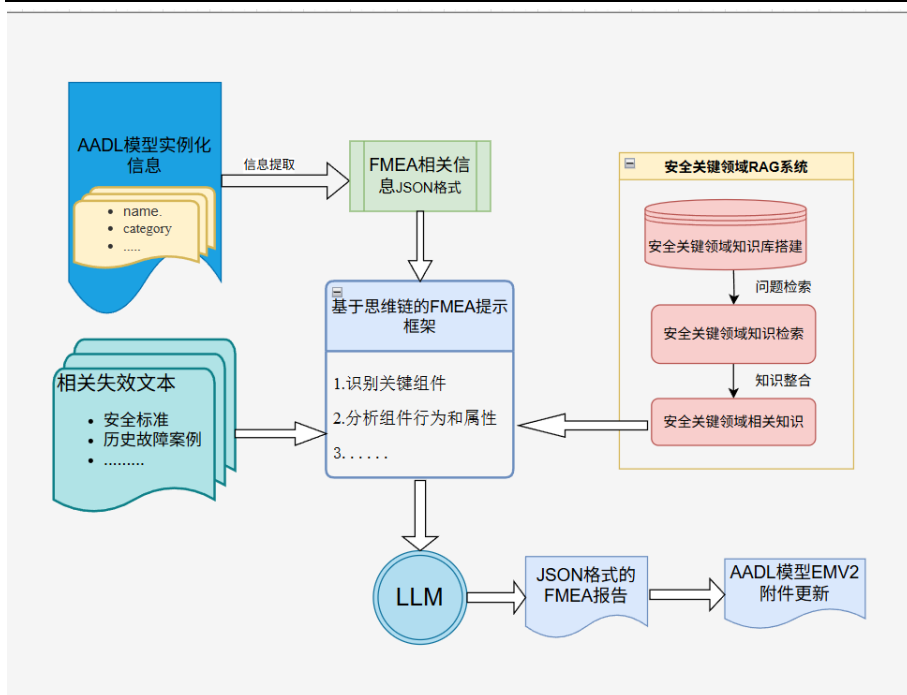


图 3.1 总体框架

批注 [A1]: 字体字号查一下

3.2 基于层级遍历的AADL模型信息压缩方法

在安全关键软件的建模与分析过程中，AADL 提供了强有力的结构化系统建模能力。通过 AADL 的系统实例化（SystemInstance）可以详细描述系统的组件层级、连接关系以及属性。然而，这种建模在生成实例化模型后，往往伴随着巨大的冗余信息，特别是在多层次嵌套和大量连接的复杂系统中更为显著。由于大语言模型（LLM）上下文长度受限，直接将这些信息输入进行故障推理分析，容易导致输入过长、信息冗余导致结果噪声等问题，从而影响大语言模型输出的 FMEA 报告质量。因此，有必要对 AADL 实例化模型进行预处理与压缩，提取对安全分析最有价值的结构信息。

本节提出一种基于层级遍历的 AADL 模型信息压缩方法，通过结构化提取、冗余剔除与路径识别等技术手段，精简了模型数据，确保了输入大语言模型的信息更聚焦于 FMEA 推理，从而优化大语言模型生成 FMEA 报告的质量并降低上下文输入负担。

3.2.1 层级遍历压缩方法的设计思路

一个完整的 SystemInstance 实例不仅包含组件实例(system、process、thread、device)，还包括它们之间的层次关系、连接通道（端口、总线及其类型）以及大量附带属性（性能、调度、可靠性、安全）。然而，对于大语言模型进行 FMEA 分析而言，只有能够反映组件失效模式、失效传播路径和故障概率的关键信息才有用，为此需要提取出 SystemInstance 实例中的相关信息，去除冗余信息。SystemInstance 实例信息与 FMEA 信息对应关系如错误!未找到引用源。表 3.1。

表 3.1AADL 与 FMEA 对应信息

SystemInstance 实例信息	FMEA 对应信息项	说明
name	组件名称（直接对应）	名称明确区分每个组件
category	失效模式（间接对应）	推出可能出现的失效模式集
parent	失效影响（通过层级关系推断）	分析失效组件中的传播
连接名称	失效影响	识别失效传播的通路
连接类型	失效原因	推出下层失效原因
属性	风险等级评估	反应失效的失效风险

为了达到上述目标，本文提出基于层级遍历的动态压缩方法，具体分为以下几个关键步骤：

（1）系统实例遍历与路径标识构建

首先从顶层的 SystemInstance 发起一次深度优先搜索，递归地去遍历每一个子组件，在遍历的过程中动态生成可唯一标识该组件位置的 pathId 字符串，从而将复杂的 AADL 树状结构展平为可索引、可追踪的平面列表。

具体而言，遍历过程从最上层的系统节点开始，初始化 pathId 为其名称，如“MainSystem”。随后对该系统的每个直接子组件递归调用遍历函数。在进入下一层级时，通过“→”符号将父级的 pathId 与当前组件名称拼接。这种拼接机制无需额外存储层级索引，pathId 自身即可完整表达节点位置，同时通过字符串即可直观还原从根节点到当前节点的完整层次结构。

（2）关键信息提取

为确保压缩后的数据高效并且能够聚焦于安全分析需求，因此在遍历到每一个组件时，提取以下的核心信息：

① 名称与类别：记录组件的唯一标识及其类型，用于明确组件的身份和功能角色，在 FMEA 中对应 Component Name，并辅助推断可能的失效模式。

② 父子关系：通过 pathId 记录组件的直接父级节点，确保层次结构的清晰表达，用于支持识别失效的传播路径。

③ 连接关系列表：列举与父级或同层组件的连接，每条记录包含连接名称及连接类型，为 FMEA 中的 Cause 与 Effect 提供失效传播通道信息。

④ 简化属性集：仅保留与失效传播或故障建模直接相关的属性，剔除默认值或空值条目，并将其转换为统一的文本格式，以降低数据冗余并提升处理效率，为 FMEA 中进行失效评估提供依据。

（3）生成 JSON 文件

在完成信息提取以及冗余消除工作之后，所生成的组件数据集采用 JSON 格式实施结构化存储，以此保证数据有良好的可读性与兼容性，JSON 格式凭借标准化的键值对结构，将 AADL 模型的层次关系以及核心信息完整留存下来，为大语言模型开展故障推理分析给予了高效的数据输入。生成的 JSON 格式如错误!未找到引用源。。

```
{
  "systemName": "DCA_tier1_Instance",
  "componentCount": 3,
  "components": [
    {
      "componentName": "DCA_tier1_Instance",
      "category": "system",
      "parent": "",
      "connections": [
        {"name": "sensor1in -> iop.sensorin1", "type": "portConnection"},
        {"name": "sensor2In -> iop.sensorin2", "type": "portConnection"},
        {"name": "iop.senseout1 -> app.samplingin1", "type": "portConnection"},
        {"name": "iop.senseout2 -> app.samplingin2", "type": "portConnection"},
        {"name": "app.controlout -> iop.controlin", "type": "portConnection"}
      ],
      "properties": [
        {"name": "MIPSBudget", "value": "2.0"}
      ]
    }
  ]
}
```

图 3.2 压缩后的 AADL 模型 JSON 结构

通过上述 JSON 生成流程，复杂的 AADL 模型被转化为精简、结构化的数据格式，不仅满足了高效性和可追溯性的需求，还为大语言模型的故障推理分析提供了高质量的输入支持，保证了输出 FMEA 报告的质量。

3.2.2 信息压缩方法伪代码

依据上述层级遍历压缩方法的设计想法，在本节中会给出此方法具体的实现逻辑，借助构建深度优先的递归遍历机制，再结合路径标识生成以及关键信息提取策略，达成对 SystemInstance 里关键信息的高效压缩和结构化表示，下面的伪代码详尽描述了这的主要步骤与操作流程，有利于后续的程序实现和逻辑验证。。。

批注 [A2]:

模型信息压缩方法

```
Input: a AAXL2 SystemInstance file (rawModel)
Output: compressed_model.json
01 compressedList ← []
02 for each comp in rawModel.getComponents() do
03     // 构造组件唯一路径标识
04     if comp.parentPathId ≠ "" then
05         pathId ← comp.parentPathId + "→" + comp.name
06     else
07         pathId ← comp.name
08     end if
10     // 初始化压缩记录
11     record ← {
12         pathId:    pathId,
13         name:      comp.name,
14         category:  comp.category,
15         parent:    comp.parentPathId,
16         connections: [],
17         properties: {}
18     }
20     // 提取连接信息
21     for each conn in comp.connections do
22         record.connections.append({ name: conn.name, type: conn.type })
23     end for
25     // 保留与失效分析相关的属性
26     for each prop in comp.properties do
27         if prop.name in {"failure_rate", "MTTF", "exception_type", "reliability_class"}
28             record.properties[prop.name] ← toText(prop.value)
29         end if
30     end for
32     compressedList.append(record)
```

3.3 基于 RAG 的安全关键领域知识增强

本节将详细介绍如何通过构建安全关键软件知识库并结合 RAG 机制实现知识检索与注入，从而增强生成模型在安全关键领域的应用能力。

3.3.1 安全关键知识库数据集构建

第一阶段首先需要进行安全关键知识库数据集的构建，核心目标是为 RAG 机制提供高质量的安全关键领域知识支持，构建面向安全关键垂直领域的大模型，确保其能够更精确的理解和处理安全关键领域的专业术语和上下文，提升在专业领域的适应性，生成比通用大模型更高质量的报告。

在考虑安全关键领域的适用性以及标准性，本文选取了来自航空工业以及汽车工业两大安全关键领域，并定义了相关性程度，依照相关性进行文档选取，最后对文档进行知识库搭建。

（1）相关性定义。

高相关性：文档直接针对安全关键系统的 FMEA 分析，给出了详细的失效模式提取方法、分析流程以及实践指南，适用于航空航天或汽车工业的安全关键软件设计，这样的文档与失效模式提取任务匹配度很高，会优先被纳入知识库。

中相关性：文档涉及安全关键系统的设计、分析或者验证，不过并没有直接聚焦于 FMEA 或失效模式提取，而是提供了支持性的指导或者相关方法。此类文档在特定场景下对 FMEA 任务能起到辅助作用，会根据具体需求被纳入知识库。

低相关性：文档涉及安全关键领域的一般要求或者管理规范，然而与 FMEA 的直接关联性比较弱，这类文档对目标领域的价值相对较低，只能作为补充参考。

数据集文档选取时首要考虑的要求便是相关性，依据上述所给出的相关性定义，本文系统地筛选标准文档，以此保证知识库内容和安全关键领域的失效模式提取任务达成高度匹配，为后续的文本拆分以及知识库构建奠定基础。

（2）文档选取

基于相关性定义，本文按照相关性程度从高到低，选取了以下标准文档作为知识库数据集的核心内容，覆盖航空工业和汽车工业两大领域：

航空工业：

批注 [A3]: 航空工业领域选取了如下标准文档

《故障模式、影响及危害性分析指南 GJB/Z 1391-2006》：高相关性。由北京航空航天大学可靠性工程研究所等起草，提供了系统、设备、组件和过程的 FMEA 分析方法，详细描述了失效模式识别、影响评估和危害性分析的流程，适用于航空航天系统的安全关键分析。

《军用软件安全性设计指南 GJB/Z 102A-2012》：中相关性。由中国人民解放军总装备部起草，提供了军用软件生命周期中安全性分析的指导，涵盖设计、开发、测试和维护阶段的安全性要求，可间接支持软件失效模式提取。

《军用软件安全性分析指南 GJB/Z 142-2004》：中相关性。提供了军用软件安全性分析的具体方法，适用于软件失效模式的识别和评估。

《系统安全性通用大纲 GJB 900-90》：低相关性。由航空航天工业部 301 所起草，规定了军用系统安全性的一般要求，涵盖管理、设计、验证等内容，作为背景知识补充。

汽车工业：

《SAE J1739》：高相关性。由国际汽车工程师学会（SAE）发布，专注于汽车设计和制造过程中的 FMEA 实践，提供了详细的失效模式识别、影响评估和纠正措施推荐方法，适用于汽车工业的安全关键系统。

《SAE ARP5580》：高相关性。虽然主要针对非汽车应用（如航空航天、工业系统），其 FMEA 方法（如功能性、接口和详细 FMEA）对汽车工业的安全关键软件分析具有参考价值。

通过系统的筛选和分级，确保了知识库中包含航空航天及汽车领域 FMEA 的核心方法论，为后续的文本拆分、向量检索与大模型知识注入奠定基础。安全关键领域 FMEA 知识的样例如下：

FMEA 知识样例

Failure Modes and Effects Analysis (FMEA) is a systematic procedure to identify potential failure modes in a system, analyze their consequences on system operation, classify their severity, and recommend corrective actions to eliminate or mitigate unacceptable effects. It encompasses functional, interface, and detailed analyses, applied to hardware, software, and processes, ensuring reliability and safety throughout the design and production stages (SAE ARP5580, Section 1).

3.3.2 RAG 机制的知识检索与注入

（1）本文使用 Lanchain-Chatchat 框架来实现 RAG，具体流程如图 3.3 所示 AG 具体流程

本地知识读取及处理

批注 [A4]: 加序号

加载文件：加载上文所选安全关键领域知识文件，如 SAE ARP5580。

读取文件：读取加载进来的文件内容，把其中冗余的注释去除掉，将知识文件转变为纯文本格式。

文本切片：按照段落规则进行文本切片，确保每块保留语义完整性，如将 FMEA 方法按章节分割。

文本向量化及存储

文本向量化：采用轻量级预训练模型（如 ll-MiniLM-L6-v2、paraphrase-MiniLM-L6-v2）将文本分块编码为高维数值向量，确保语义特征的有效捕获。

存储：基于 FAISS 框架构建高效索引结构，将文本向量存入数据库，支持快速近似最近邻检索。

问句向量化及相似性匹配

查询编码：使用与文本分块相同的 Embedding 模型，将用户查询映射至同一向量空间。

相似度计算与检索：通过 FAISS 索引计算问句向量与文本向量的余弦相似度或欧氏距离，筛选相似度最高的 Top-K 个候选文档。本文采用余弦相似度进行计算，余弦相似度能够高效衡量文本语义相似性，同时避免因文本长度或向量大小差异导致的干扰。

其计算公式如下：

$$\text{cosine_similarity}(A, B) = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}} \quad (3.1)$$

上下文增强的 prompt 构造

将 Top-K 文档作为领域知识上下文，与用户查询按模板整合为结构化 Prompt，确保输入符合大语言模型的长度与语义要求。

大语言模型响应

将构建的 prompt 输入大语言模型，利于大语言模型的综合推理能力输出具有逻辑严谨、领域适配的答案，如基于 SAE ARP5580 的 FMEA 回答。

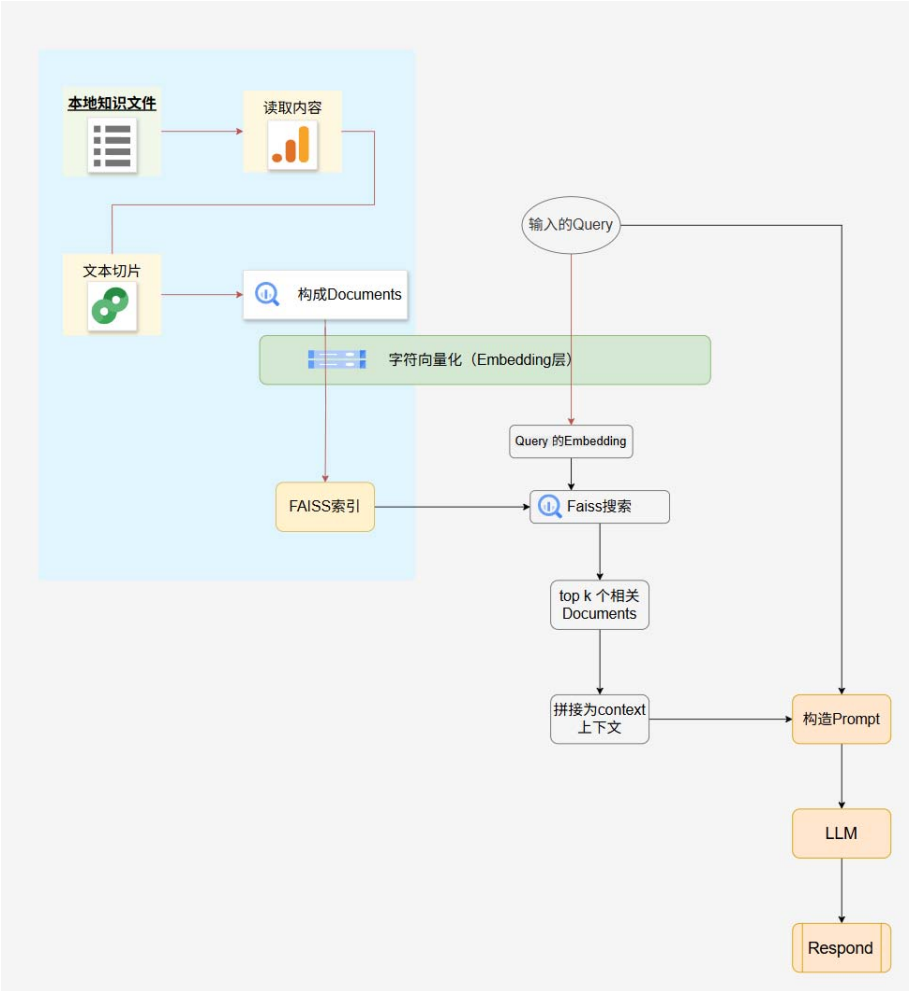


图 3.3 基于 Langchain-Chatchat 框架的 RAG 流程图

(1) RAG 系统性能优化

针对检索到相关内容却未被生成模型利用问题

在 RAG 系统中，Top-K 参数（即 k 值）直接控制检索模块返回的最相似文档数量。如设置 $k=5$ 时，系统只会选取前 5 条最相关的结果；当 k 过小时，检索覆盖不足，关键证据可能被遗漏，如缺少 FMEA 的核心定义等，导致生成模型因上下文信息匮乏而无法充分推理回答；而当 k 过大时，则会携带大量冗余段落，干扰模型聚焦核心信息，降低生成质量。

另一方面，不同大语言模型对上下文窗口有严格的长度限制，当 k 值过大以至于拼接后的 Prompt 超出模型最大上下文容量时，后续内容会被截断或丢失，显著影响回答的完整

批注 [A5]: 改成句子段落

性与连贯性。鉴于所使用的语言模型最大上下文长度存在一定限制，一般处于 8k 至 128k 的范围，本研究经过综合考量后，最终把 k 值设定为 5，如此一来，可在覆盖安全关键领域知识以及多余信息噪声干扰之间找到最为适宜的平衡状态，以此保障 RAG 系统检索到安全关键领域知识时有较高的质量。在安全关键领域当中，像 FMEA 分析这类情况，切片工作要要保证关键方法论维持完整的状态，可提升检索的准确性。

针对 RAG 检索出错问题

RAG 系统的检索性能在很大程度上依赖于文本分块策略，要是把块大小设定过小，虽说局部匹配会更精准一些，然而却会致使上下文信息出现割裂的情况，丢失关键内容，要是块大小设置得过大，尽管上下文能保持完整，但也带入了过多的噪声，容易让检索结果偏离问题的意图。经过多组对比实验发现，把块大小设置在 200 至 400 tokens 这个区间，可在保留充足上下文和减少冗余噪声之间达成最佳的平衡，以此提升检索的准确性以及系统的整体稳定性，在安全关键领域，像是 FMEA 分析，切片要保证关键方法论完整保留，这样才能有效为大语言模型注入相关知识。

基于上述工作,RAG 系统能够基于询问句检索出最相关的安全关键领域 FMEA 知识。
RGA 检索到的安全关键领域知识如表 3.2:

表 3.2RAG 系统查询句与对应知识

询问句	FMEA 相关知识
FMEA 的定义	FMEA 是识别系统失效模式、评估影响和严重性、推荐纠正措施的方法（SAE ARP5580, 第 1 节；GJB/Z 1391-2006, 第 4.1 节）
FMEA 的类型	包括功能性、接口、详细和过程 FMEA，适用于设计和制造（SAE J1739, 第 3.2 节；GJB/Z 1391-2006, 第 5.2 节）
FMEA 文档要求	包含系统描述、失效模式、影响分析、纠正措施（SAE J1739, 第 5.1 节；GJB/Z 1391-2006, 第 7.3 节）
FMEA 步骤	识别失效模式、分析影响、评估严重性、推荐纠正措施（GJB/Z 1391-2006, 第 6.2 节；SAE ARP5580, 第 6 节）。

基于上述工作，实现了安全关键领域 FMEA 知识的结构化梳理以及高效检索，给大语言模型在专业场景中提供了可靠的知识支持，借助 RAG 机制，模型在面对具体问题时可调用最为相关的行业标准和方法论，切实提高了大语言模型生成 FMEA 任务时的专业性、准确性以及生成质量。

3.4 基于思维链的 FMEA 提示框架

为提高 FMEA 报告生成的准确性与可靠性，本文提出一种基于思维链（Chain-of-Thought, CoT）推理机制的 FMEA 提示框架。

批注 [A6]: 行间距

3.4.1 思维链推理逻辑的设计

大语言模型所有的推理能力和 **prompt** 的设计之间存在着很强的相关性，**prompt** 作为用户跟模型进行交互时的核心输入内容，其质量对于模型输出效果有着直接的决定性作用，并且还会对 RAG 框架生成上下文知识的组织以及利用情况产生影响，传统的 **prompt** 设计一般依靠人工经验，缺乏系统性又缺少针对性，当面对特定领域中的复杂问题时，容易因为表达不够准确而致使输出效果不理想。

失效模式和影响分析是一项涉及系统结构与潜在故障复杂关系的系统性工程方法。其分析过程通常包括组件识别、功能行为分析、故障模式推断和风险评估等多个子步骤。传统 **prompt** 直接询问大语言模型一键生成 FMEA 报告，会导致模型因缺乏明确推理路径而忽略关键信息或出现逻辑跳跃等问题，影响生成报告的准确性与可解释性。

为了可提升生成的准确性，我们提出了一种借助思维链推理机制构建的 FMEA 提示框架，此框架是凭借在 **Prompt** 里面设计有分步逻辑的推理流程，以此引导模型逐步去模拟人工分析的思维路径，提高生成 FMEA 报告的完整性以及可信度。具体推理步骤如下：

- (1) 关键组件识别：首先解析 AADL 模型结构，确定与故障情景相关的系统模块和功能单元。如提取子系统、硬件单元和接口等关键信息，为后续分析聚焦对象。
- (2) 行为解析：对识别出的组件，说明其在正常状态下的功能和作用，如输入输出、工作流程等。通过明确正常行为，有助于思考异常后果。
- (3) 故障模式生成：针对每个组件结合失效案例，推断可能发生的故障类型及其产生原因。例如，硬件失效、通信中断或逻辑错误等。同时考虑故障时对系统功能的影响。
- (4) 风险优先级评估：对每种故障模式，综合评估严重度、发生概率和可检测度，然后计算风险优先级数 $RPN = \text{严重度} \times \text{发生度} \times \text{可检出度}$ 。此步骤要求模型细化每个指标评分，并得出最终 RPN。

通过显式的推理引导，大模型依据上述四个推理阶段来逐步开展思考，在生成最终结果之际，能保证输出每一步的逻辑推理过程，以此提升回答的可信度。

3.4.2 FMEA 提示模板的构造

为了使大模型可依照上述逻辑输出 FMEA 报告，本文精心设计了与之相应的 **Prompt** 模板，这个模板是从任务设定、输入组织、推理引导、示例引导以及输出约束这五个方面来进行系统构建的，其目的在于激发模型有多步推理的能力，同时保障输出结果的准确性以及结构的一致性。

首先，需要在提示词中设置系统提示。系统提示需要明确模型角色与任务目标，有助于增强模型对任务语境的理解。系统提示如下：

系统提示

你是一名系统可靠性工程师，请对下列系统模型和故障描述进行失效模式影响分析 (FMEA)

其次 Prompt 输入结构被划分为两部分：系统模型（AADL 结构化信息）与特定故障文本。提示中要求模型基于这两类输入信息，按步骤逐项完成分析，从而增强输入信息的利用效率。

同时在提示词中引入上文设计的思维链推理逻辑，进行大语言模型引导，明确列出分析流程。基于思维链的提示如下：

基于思维链的提示

"要求按照下面步骤进行一步步推理："

1.识别关键组件："分析 AADL 实例化模型中的所有系统、进程、线程、处理器、内存、设备等，列出所有组件，包括子组件和嵌套组件。通过识别组件之间的连接、接口和依赖关系来分析组件的逻辑关系和依赖关系。

"2.分析组件行为和属性："对于每个组件，识别其行为定义，包括动作、状态机、活动等。对于没有行为的组件，识别其关键属性和约束。

"3.生成故障模式："对于有行为的组件：对于每个行为，考虑其失效情况（例如，行为无法执行、执行错误、执行时机错误等）；对于状态机，考虑状态转换失败、卡在某个状态等。"对于没有行为的组件：对于每个关键属性，考虑其失效情况（例如，属性值过高、过低、不稳定、丢失等）。

4.生成 FMEA 条目："对于每个故障模式，分析其可能的原因，预计会造成的影响，评价其严重程度（1-10）。"

当思维链提示词构建好之后，本文接着引入 Few-shot 示例，以此提高大语言模型对 FMEA 任务格式、推理步骤以及表达风格的模仿能力，借助精心设计的三个典型示例，模型在处理实际任务时可使分析流程与输出结构相匹配，提高推理的准确性和一致性，为了达成结果的结构化解析以及下游处理，Prompt 里额外添加了输出约束，规定模型只能以 JSON 格式生成 FMEA 报告，保证内容标准化、可解析且可拓展。将上述 Prompt 和经 RAG 机制检索获得的安全关键领域知识一同输入大语言模型，完成了端到端的 FMEA 报告生成流程。以下为生成的 FMEA 报告示例：

```
{
  "name": "DSP的GPIO寄存器",
  "failure_modes": [
    {
      "mode": "通信功能不能实现",
      "cause": "DSP的GPIO寄存器异常",
      "effect": "可能导致外部设备通信中断，影响整体系统协调",
      "severity": 8,
      "occurrence": 5,
      "detection": 6,
      "RPN": 240,
      "optimization_method": "通信功能异常时上报飞机",
      "optimization_effect": "快速定位问题并采取应急措施",
      "Recommended Actions": "增加GPIO状态监控和冗余通信路
径",
      "New RPN": 120
    }
  ]
}
```

图 3.4 生成 JSON 格式 FMEA 示例

3.5 基于 FMEA 报告的 EMV2 附件扩展

EMV2 是用于系统可靠性建模与分析的标准架构语言）的扩展模块，主要用于描述系统错误传播、容错机制及安全属性。为实现从故障分析到系统架构模型的自动化映射，并提升安全分析的一致性与效率，本节提出了一种基于 FMEA 报告的 EMV2 附件转换方法。该方法将大语言模型生成的 FMEA 报告，拓展至 AADL 的 EMV2 附件，实现端到端的模型更新与校验。

3.5.1 元素映射方法

为实现 FMEA 报告到 EMV2 附件转化，需对故障模式、故障影响、检测能力与风险等级和容错策略等核心元素，逐一映射至 EMV2 的相应结构。下面从故障模式、失效状态、传播路径、风险等级以及恢复与容错五个方面，详细说明映射规则。

(1) 故障模式映射至错误类型

EMV2 附件中内置了一个“错误类型库（Error Library）”，该库汇总了常见的系统故障模式，形成了可嵌入模型的严格分类本体。将 FMEA 报告中每一条故障模式抽取并标准化命名，映射为 EMV2 中唯一的 Error Type。该映射过程可依托于 EMV2 的错误类型库。另外在映射过程中，应保持故障模式命名与 FMEA 报告中的语义一致性，以实现模型生成与分析结果的准确对应。

(2) 失效状态映射为组件行为状态

在 EMV2 中，组件的运行状态可通过状态机的形式进行建模，如 Operational、Failed、Degraded 等。FMEA 报告中的故障影响往往会导致系统功能丧失或性能下降，这类影

响应映射为组件的错误状态。当某一故障模式被触发，组件会由 Operational 状态转换至 Failed 或 Degraded 状态。在 EMV2 的 component error behavior 中，通过 transitions 定义状态转换路径，并由 error event 触发。通过这种映射方式，能够明确故障与系统功能丧失之间的关系，为进一步建模容错机制和安全响应提供基础。

(3) 故障影响映射为错误传播路径

FMEA 报告中通常包含对故障扩散影响的定性描述，特别是在多个组件间存在依赖关系时，单点故障可能引发系统级连锁反应。EMV2 支持通过 in propagation 和 out propagation 语义建模这种跨组件的故障传播路径。将故障影响映射为错误传播路径可遵循如下规则：

若组件 A 的故障导致组件 B 接收错误输入，则在 A 中定义 out propagation，在 B 中定义对应的 in propagation，传播路径与错误类型绑定，实现类型一致性传播，结合 guard 条件限定传播条件，实现传播筛选与抑制。

这种传播建模机制使得 FMEA 中描述的影响链条得以形式化表达。

(4) 检测能力与风险等级映射为属性特征

FMEA 报告中包含多个量化指标，用于评估系统面对故障的安全风险程度，其中包括故障的严重度、发生率、检测性以及由三者计算得到的风险优先数。这些指标可直接映射为 EMV2 模型中的属性特征，用于支持故障严重性评估与风险排序。

(5) 恢复与容错机制映射为错误处理行为

FMEA 报告中提出的优化建议与容错措施，应映射为 AADL 组件的恢复行为或冗余机制建模。在 EMV2 中，可通过如下方式实现：

在状态机中添加 Recovered 状态，并定义从 Failed 到 Operational 或 Degraded 的恢复路径，另外利用 Guard 或条件转移控制恢复触发条件。另外将 FMEA 中建议的优化措施作为自定义属性，如 FMEA_Properties::Optimization_Method 附加至对应错误事件，增强设计与分析的可追踪性。

映射表格如表 3.3：

表 3. 3FMEA 与 EMV2 映射表

FMEA 要素	EMV2 映射结构	说明
故障模式 (Failure Mode)	Error Type (错误类型)	每个故障模式标准化命名为唯一 Error Type，依托 Error Library 建模
失效状态 (Failure State)	Error State (错误状态)	映射为组件运行状态（如 Operational、Failed、Degraded），用于描述故障发生后的行为变化
故障影响 (Effect)	Error Propagation (错误传播路径)	描述故障扩散路径，支持 in/out 定向传播，结合 Guard 可控传播
检测能力与风险等级	Hazards 属性特征	用于支持故障严重性评估与风险排序
优化建议 (Recommendations)	Property	化建议可映射为自定义属性

3.5.2 基于大语言模型的转换实现

为实现上述 FMEA 报告到 EMV2 附件的映射自动化转换，本文给出了一种基于大语言模型的自动转换方法，用于解析 FMEA 报告，并按照映射规则生成符合 AADL 规范的 EMV2 附件代码。该工具包括以下核心流程：语义解析、映射推理与代码生成。工具支持从 FMEA 报告中提取关键信息，依据映射规则将其转化为 EMV2 结构，并生成更新后的 AADL 代码。如图 3. 5 所示。

(1) 语义解析

在输入结构化 FMEA 数据后，LLM 首先对每条记录进行语义解析，识别其中的组件名称、故障模式、影响路径、风险等级以及优化建议等核心要素。同时，结合 AADL 模型中已定义的组件结构信息，模型能够自动构建上下文语义，识别各故障对应的系统结构位置。

(2) 元素映射模块

基于预设的提示工程，LLM 使用前置引导语对各 FMEA 字段进行格式化映射。该阶段根据 3.5.1 节所定义的映射规则完成，映射过程确保语义一致性。

(3) 代码生成模块

在完成字段到结构语义的转换后，LLM 进一步生成符合 AADL 语法规则的完整代码片段。生成内容包括组件定义、EMV2 附件、错误类型库引用、状态机、传播路径及属性声明等。输出可嵌入至 AADL 模型中，被 OSATE 等工具识别与分析。

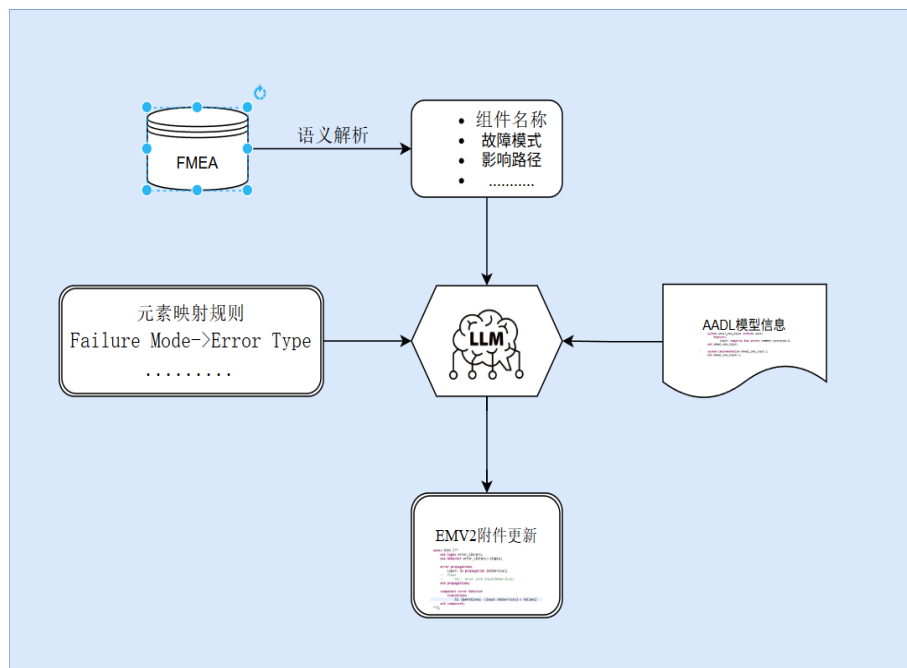


图 3. 5FMEA 转换 EMV 流程图

3.6 本章小结

本章提出了基于大语言模型的 FMEA 生成方法，旨在通过自动化技术提升安全关键领域的失效模式影响分析效率。首先，介绍了生成 FMEA 方法的总体框架，其次使用层级遍历的 AADL 模型信息压缩方法，精简了模型数据，确保了输入大语言模型的信息更聚焦于 FMEA 推理。接着，采用 RAG 机制增强安全关键领域的知识库，确保了 FMEA 分析中能够准确地融入行业相关背景知识。

在 FMEA 提示框架方面，提出了基于思维链的推理逻辑用于解决 FMEA 任务，构建了适用于大语言模型的提示模板，从而提升了模型的推理能力与生成质量。随后，介绍了基于 FMEA 报告的 EMV2 附件扩展方法，通过详细的元素映射规则，实现了从 FMEA 报告到 EMV2 附件的自动化转换，支持了安全设计的进一步发展。

本章从 FMEA 生成方法的框架设计、专业知识的增强，到基于报告的模型更新和校验，构建了一个端到端的自动化流程，提供了高效、智能的工具来提升安全关键系统的安全设计与分析能力。

第四章 框架效果评估与对比分析

批注 [A7]: 段落格式不对

本章为验证上文方法的效果和可行性，进行了三个实验对比分析，证明上文方法的可行性。实验分别包括 AADL 压缩效果验证、RAG 安全关键知识获取验证以及提示框架效果验证，从不同维度验证方法的实用性与高效性。

4.1 AADL 压缩效果验证

为了验证上文的所提出层级遍历压缩方法的通用性与有效性，本文进行了 AADL 压缩效果验证。

4.1.1 数据集选择

为全面验证 AADL 模型压缩方法的通用性和有效性，本文选取了四个具有显著差异化特征的 AADL 实例模型作为实验基准，涵盖航空、嵌入式控制、资源调度等典型安全关键领域。数据集选择模型特性如下：

(1) ARP4761 航空安全模型

领域背景：该模型基于航空安全标准 ARP4761 构建，模拟民用飞机航电系统的故障传播与安全性评估流程。

结构特性：包含 3 层嵌套（系统→子系统→设备），组件间通过故障模式库关联。

(2) Ejection Seat 弹射座椅控制系统

领域背景：战斗机弹射座椅的嵌入式实时控制系统，涵盖座椅点火时序、姿态调整与应急协议

结构特性：基于状态机响应飞行员指令、环境传感器事件

(3) Resource-Budget 资源分配模型

领域背景：卫星通信系统的资源预算分配模型，优化多任务负载下的资源利用率。

结构特性：定义了细粒度资源占用规格，并且持运行时资源重分配策略

(4) Speed-Regulation 速度调节控制回路

领域背景：汽车发动机电控单元的闭环速度控制模型，实现 PID 算法与执行器联动。

结构特性：闭环反馈结构，既传感器→控制器→执行器→物理模型迭代

这四个模型分别代表了中等嵌套、事件驱动、资源密集和控制回路导向的多种系统架构，覆盖了不同规模、不同关注点的 AADL 应用场景，为压缩方法的通用性和有效性验证提供了全面测试基准。

4.1.2 实验结果

实验结果证明了所提出的层级遍历压缩方法在 AADL 模型压缩中既保证了关键信息的提取，又保持了鲁棒性。具体而言，该方法保留了组件名称、失效模式及风险等级等核心信息，确保了模型的结构和安全属性完整，为 FMEA 分析提供的必要的信息。同时，实验在不同规模和类型的模型上实现了超过 90%的压缩比，压缩后文件大小控制在 50 KB 左右下，保留节点比例低于 20%，体现了方法的通用性和稳定性，为安全关键领域的应用提供了可靠支持。

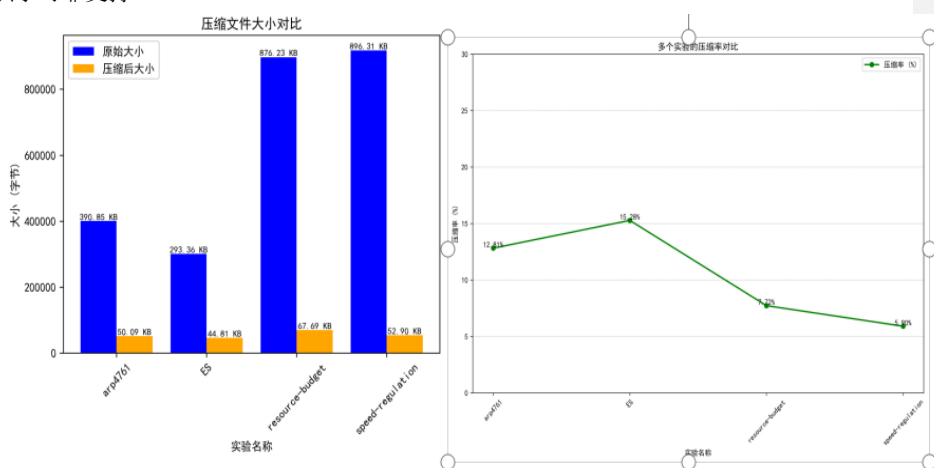


图 4.1 压缩实验对比结果

这种方法通过高效的压缩技术，显著降低了上下文输入的冗余量，从而极大优化了数据处理效率。与此同时，为大语言模型生成 FMEA 报告提供了轻量且高质量的数据基础，使得分析过程更加精简和精准。这种轻量化处理方式不仅减少了计算资源的消耗，还提升了模型在复杂分析任务中的响应速度和可靠性。

4.2 RAG 安全关键知识验证

本节为验证上述 RAG 系统在检索安全关键知识时的准确性与可靠性，设计了针对性实验。实验通过对比多个轻量化嵌入模型在安全关键领域知识检索能力上的表现，评估其性能差异。结果表明，RAG 系统在不同模型配置下均能高效检索相关知识，且在语义准确性和可靠性上表现出色，为安全关键系统的知识支持提供了有力保障。

4.2.1 数据集构造

为验证 RAG 系统在安全关键领域失效模式与影响分析知识检索中对上下文信息获取的准确性与可靠性，本文设计并执行了评估实验。实验的对比数据集构造如下：

(1) 数据来源与构建：数据集基于航空航天系统安全分析的相关文献以及工业 FMEA 标准，全面涵盖结构信息、功能特性、故障影响等多层级内容。通过整合权威资料，构建了包含问答对的集合，作为检索-生成任务的基础。数据集设计注重多维度信息覆盖，确保能够反映 FMEA 分析中的典型场景和关键需求。

(2) 问答对生成：基于构建的知识库，提取并整理了结构化问答对，涵盖 FMEA 流程的核心要素。问答对由领域专家参与验证，确保数据内容的准确性与专业性，为后续实验提供可靠的基准。

(3) 查询集准备：为模拟实际应用场景，选取了覆盖 FMEA 核心任务流程的代表性问题，例如 “How to make FMEA?”。这些问题由领域专家标注标准答案，作为评估 RAG 系统检索与生成性能的参考依据。具体示例如下表 4. 1：

表 4. 1RAG 查询句对应标准答案

询问句	标准回答
How to make FMEA ?	(1) Defining system functions and interfaces, (2) Identifying failure modes,(3) Analyzing effects across system levels, (4) Assessing severity and detection, (5) Identifying compensating provisions,and (6) Documenting in worksheets
What are the steps to analyze failure effects in FMEA?	(1) Identifying potential failure modes, (2) Evaluating effects on system performance, (3) Assessing propagation across subsystems, (4) Determining criticality, and (5) Recording findings in FMEA documentation.
How to assess severity in an FMEA process?	(1) Defining severity criteria based on safety impact, (2) Assigning severity ratings (e.g., 1-10 scale), (3) Considering operational consequences, (4) Validating with domain experts, and (5) Updating risk prioritization accordingly.

通过上述步骤，成功构建了一个结构完整、内容丰富的实验对比数据集，为后续 RAG 系统性能评估提供了强有力的支持。

4.2.2 实验指标设计

为了对 RAG 系统的检索性能作出评估，本文运用 Recall@K、Weighted Recall@K 以及 NDCG@K 这三个指标，量化检索的准确率以及结果排序质量，借助 BERTScore-F1 指标来对检索与生成结果的一致性进行量化评估，另外为衡量生成知识与标准知识之间的语义相似度，本文引入了 Semantic Similarity Score (SSS)指标。该指标是基于 Sentence-BERT

模型的，它会把生成文本和标准答案编码成为语义向量，并且凭借余弦相似度来计算两者语义的接近程度，其值范围是[0, 1]，数值越高也就意味着语义越相似。五项指标的定义如下：

（1）Recall@K（前 K 召回率）：衡量检索系统在前 K 个结果中包含的相关文档比例，反映检索覆盖能力。其计算公式如下：

$$\text{Recall@K} = \frac{|\text{Rel} \cap \text{Top-K}|}{|\text{Rel}|} \quad (4.1)$$

（2）Weighted Recall@K（加权前 K 召回率）：在 Recall@K 基础上，考虑相关文档的权重使得更相关的内容，占比更高，从而更精准评估检索效果。其计算公式如下：

$$\text{Weighted Recall@K} = \frac{\sum_{d \in \text{Top-K}} w_d \cdot I(d \in \text{Rel})}{\sum_{d \in \text{Rel}} w_d} \quad (4.2)$$

（3）NDCG@K（归一化折损累积增益）：评估检索结果的排序质量，考虑相关文档的排名位置，越靠前得分越高。其计算公式如下：

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}, \quad (4.3)$$

（4）BERTScore-F1（检索-生成一致性）：使用预训练 BERT 模型评估检索文档与生成回答的语义相似性，衡量生成内容对检索结果的利用程度。其计算公式如下：

$$\text{BERTScore-F1} = 2 \frac{P \cdot R}{P + R} \quad (4.4)$$

（5）SSS（语义一致性）：通过预训练的 Sentence-BERT 模型将生成文本和标准答案编码为语义向量，并计算两者之间的余弦相似度。SSS 的值范围为 [0, 1]，值越高表示语义越相似。其计算公式如下：

$$\text{SSS} = \cos(\vec{v}_{\text{generated}}, \vec{v}_{\text{standard}}) = \frac{\vec{v}_{\text{generated}} \cdot \vec{v}_{\text{standard}}}{|\vec{v}_{\text{generated}}| \cdot |\vec{v}_{\text{standard}}|} \quad (4.5)$$

通过上述五个指标，能够全面评估 RAG 系统的检索准确性、排序质量以及生成内容的语义一致性，为 RAG 安全关键知识检索效果提供量化依据。

4.2.3 实验结果

实验结果如错误!未找到引用源。和错误!未找到引用源。。实验结果表明，基于 RAG 的检索增强系统能够精准适配安全关键领域的知识需求。特别的在语义准确度匹配上表现出色，证明其能够为大语言模型注入对应的安全关键领域的知识，增强大语言模型对于安全关键领域的专业知识。

批注 [A8]: 缩进同上

批注 [A9]: 行间距

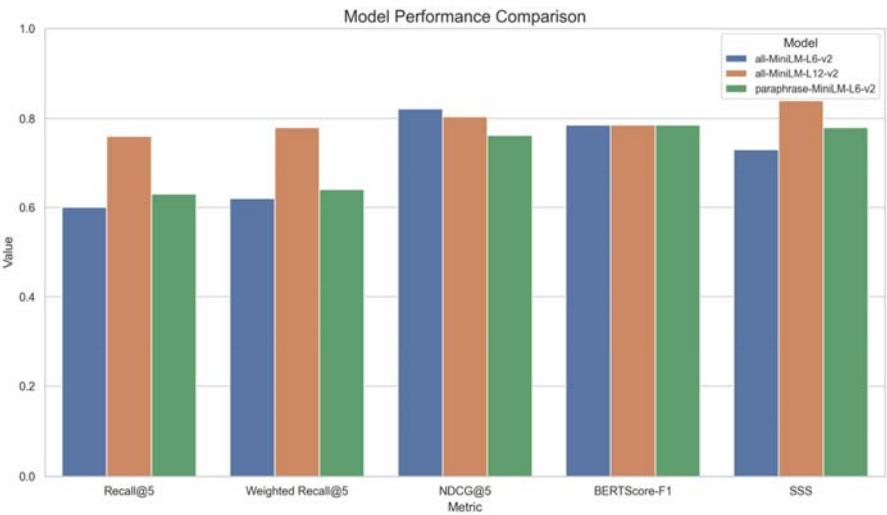


图 4. 2RAG 安全关键知识检索效果

表 4. 2RAG 安全关键知识检索效果数据

模型	Recall@5	Weighted Recall@5	NDCG@5	BERTScore-F1	SSS	加载时间 (s)	检索延迟 (s)
all-MiniLM-L6-v2	0.60	0.62	0.8213	0.7856	0.73	0.92	0.09
all-MiniLM-L12-v2	0.76	0.78	0.8039	0.7856	0.84	0.83	0.02
paraphrase-MiniLM-L6-v	0.63	0.64	0.7625	0.7856	0.78	0.75	0.01

此外，对于实验的三个轻量化模型，在安全关键领域知识的检索上都有着不错的效果，其中 all-MiniLM-L12-v2 模型表现尤为突出，其在核心性能指标上都 80%左右。同时实验结果表明，RAG 系统在检索安全关键领域知识上具有良好的鲁棒性，在不同参数的模型检索下都具有良好的知识检索效果。

4.3 提示框架效果验证

为系统评估基于思维链的 FMEA 提示工程框架在实际应用中的有效性，本节设计并进行对照实验，通过量化对比分析框架输出与传统提示方法生成效果的差异。

4.3.1 数据集构造及模型选择

（1）标准数据集构建

为验证基于思维链的FMEA提示框架的效果，本文人工构造了一套标准化的FMEA报告作为基准数据集。通过结合失效模式与故障案例制作报告模板。每条报告都包含上文提示词中的FMEA信息。总共包含121条FMEA报告。

（2）大语言模型选取

Base模型与Chat模型的选择。大语言模型按照训练目标以及应用场景，一般会被分成Base模型与Chat模型，Base模型把通用语言建模当作核心，借助海量未标注文本数据开展自回归预训练，数据来源涉及书籍、网页文章等开放领域通用文本，主要是用来学习语言的语法结构、语义关系、逻辑推理以及词汇预测等基本任务。Chat模型是在Base模型的基础上，依靠指令微调以及对话数据优化做训练，采用特定对话数据集跟指令数据集，以此提升模型对用户指令的理解能力以及在多轮对话中的表现，本研究关注安全关键领域的专业知识与内容，需要模型在多轮对话里准确理解指令并给出理想回复，选择Chat类模型来做实验。

大模型参数量的选定。在挑选大语言模型的参数量时，要全面考量任务需求、硬件资源、性能与成本的平衡情况，在实验设计当中，把模型参数量划分成中小规模模型和大规模模型，中小模型的参数为6B至13B，而大规模模型为70B以上。中小规模模型适合处理比较简单的任务，大规模模型更适合高复杂度的任务，为了验证框架的鲁棒性，本文会选取两种规模的模型。

综上，本文选用的大语言模型有Deep Seek-R1、qwen-plus、llama3.1-8B三个大语言模型。模型的详细参数如表4.3：

表 4.3 大语言模型参数表

	类型	参数量	上下文窗口长度	主要特点
Deep Seek-R1	Chat 模型	671B	128K	专注推理、数学、编码任务，鲁棒性强
qwen-plus	Chat 模型	235B	128K	支持复杂推理和对话
llama3.1-8B	Chat 模型	8B	128K	多语言对话模型，性能优异

批注 [A10]: 表格太宽

4.3.2 实验指标设计

为系统评估基于思维链推理机制的 FMEA 提示框架的有效性，本文设计并实施了对比实验，通过覆盖率、准确性、一致性和生成效率四个关键指标，对框架在 FMEA 报告生成任务中的性能表现进行了全面分析。

四个评估指标的定义如下：

覆盖率：指生成 FMEA 报告中有效条目的数量与参考标准报告中条目总数之间的比值，用以衡量模型在故障模式与功能影响识别上的全面性。

$$\text{Coverage} = \frac{|\text{Valid}_{\text{gen}}|}{|\text{Ref}|} \quad (4.6)$$

其中， $(\text{Valid}_{\text{gen}})$ 为生成报告中有效条目集合， (Ref) 为参考报告中条目总数。

准确性：指生成报告中各关键字段（如组件名称、故障模式、影响描述、RPN 值）与基准报告内容一致的比例，用于评估生成内容的正确性与语义贴合度。

$$\text{Accuracy} = \frac{|\text{Correct}_{\text{gen}}|}{|\text{Total}_{\text{gen}}|} \quad (4.7)$$

其中 $(\text{Correct}_{\text{gen}})$ 为生成报告中与参考一致的字段数， $(\text{Total}_{\text{gen}})$ 为生成字段总数。

一致性：表示在多轮生成任务中，模型输出是否严格遵循预设格式规范的比例，用于衡量模型生成结构的稳定性与可靠性。

$$\text{Consistency} = \frac{|\text{Compliant}_{\text{gen}}|}{N} \quad (4.8)$$

其中， $(\text{Compliant}_{\text{gen}})$ 为遵循格式规范的生成报告数， (N) 为总生成轮次。

生成效率：指模型从接收输入 Prompt 至输出完整 FMEA 报告所需的平均时间（单位：秒），反映提示结构在引导模型推理生成过程中的响应效率。

$$\text{Generation Efficiency} = \frac{\sum_{i=1}^N t_i}{N} \quad (4.9)$$

其中， (t_i) 为第 (i) 生成任务耗时， (N) 为总生成轮次。

本实验进行传统的直接提示与基于思维链的 FMEA 框架进行了对比。为大语言模型分别应用两种用户提示。传统直接提示直接要求大语言模型根据输入的 AADL 信息结合失效文本直接输出对应的 FMEA 报告。基于思维链的 FMEA 提示框架，则要求按照传统的

FMEA 分析过程按照步骤进行推理,最终输出对应的 FMEA 报告。其中基于思维链的 FMEA 提示框架上文已给出,直接传统提示如下:

直接 prompt

请根据所提供的 AADL 模型结构信息及其关联的故障描述文本,生成一份完整的 FMEA (Failure Modes and Effects Analysis) 报告。报告内容应包括系统中各组件的功能描述、潜在失效模式、失效后果、可能原因,以及相应的风险评估指标 (包括严重度 Severity、发生概率 Occurrence、可检测度 Detection) 和由此计算的风险优先级数 (RPN)。输出结果请采用结构化 JSON 格式,仅包含 FMEA 报告内容本身,不包含其他说明性文字。

4.3.3 实验结果

实验结果如表错误!未找到引用源。。FMEA 提示框架在覆盖率、准确性以及一致性这三个关键指标上都比传统直接 Prompt 更具优势,体现出它在提升生成质量方面的优势,在覆盖率方面,Qwen-Plus 表现最优,达到了 62.8%,相较于传统 Prompt 的 48.8%有较为十分突出的提高,Deep Seek 和 Llama3.1-8B 也分别提升到了 59.5%和 53.6%。在准确性方面,Deep Seek 表现十分突出,从 25.6%大幅提升至 77.4%,Qwen-Plus 和 Llama3.1-8B 也分别达到 61.0%和 34.3%,都比传统方式更具优势,同时在输出一致性方面,Deep Seek 以 92.3%位居 Qwen-Plus 和 Llama3.1-8B 也分别提升至 84.6%和 70.8%,FMEA 框架可有效提高输出 JSON 格式的稳定性。

表 4. 4prompt 实验结果

Prompt 类型	指标	Deep Seek	qwen-plus	llama3.1-8B
传统直接 Prompt	覆盖率	50.4%	48.8%	43.6%
	准确性	25.6%	11.0%	9.7%
	一致性	76.9%	61.5%	59.4%
	生成效率	689.95s	492.50s	584.50s
FMEA 提示框架	覆盖率	59.5%	62.8%	53.6%
	准确性	77.4%	61.0%	34.3%
	一致性	92.3%	84.6%	70.8%
	生成效率	693.70s	537.30s	653.45s

虽然在生成效率方面，由于链式提示在推理过程中引入了更多语义步骤，导致生成时间略有增加，但这一代价是有限且可控的，远远低于其在性能上的收益，不会对实际应用造成显著负担。上述实验结果显示，基于思维链的 FMEA 提示框架具有良好的性能。

4.4 本章小节

本章通过三个实验对上文提出的大语言生成 FMEA 报告方法进行了全面的对比分析，从不同维度验证了其效果与可行性，充分证明该方法的实用性与高效性。首先，AADL 压缩效果实验展示了在保持关键输入信息的同时有效降低输入复杂度。其次，RAG 安全关键知识获取实验证明了 RAG 系统在提取和处理安全关键知识时的准确性和可靠性，为大语言模型在安全关键领域提供了强有力的上下文知识支持。最后，提示框架效果实验表明，FMEA 提示框架在覆盖率、准确性和一致性上显著优于传统直接 Prompt，能够生成更高质量的 FMEA 报告。

第五章 工具实现与案例分析

本章介绍基于 OSATE 平台开发的失效模式影响分析原型工具 FMEAanalysis，并以工业界弹射座椅控制系统为例进行分析，验证本文所提方法和工具的有效性

5.1 工具设计与实现

FMEAanalysis 工具的设计和实现用于辅助用户从 AADL 模型结合失效文本实现到 FMEA 报告的端到端生成。工具总体框架基于 OSATE 平台，使用通过 Java 进行插件的开发，能够实现 AADL 实例化信息的压缩提取，并通过调用 LLM SDK 完成大语言模型调用。在实现过程中，工具使用 LangChain 框架实现交互，利用嵌入模型实现对于安全关键领域知识文档的向量化表示与相似度检索，并利用基于思维链的 FMEA 提示框架实现高效准确的 FMEA 报告生成。同时，工具界面采用模块化设计如图 5.1，包含文件输入、配置选取、提示组合、FMEA 展示等功能模块，为用户提供直观、便捷的操作，最后通过 FMEA 报告更新 AADL 模型。通过集成上述技术，FMEAanalysis 实现从 AADL 模型到 FMEA 报告的无缝转换，显著提升安全分析的效率和一致性。

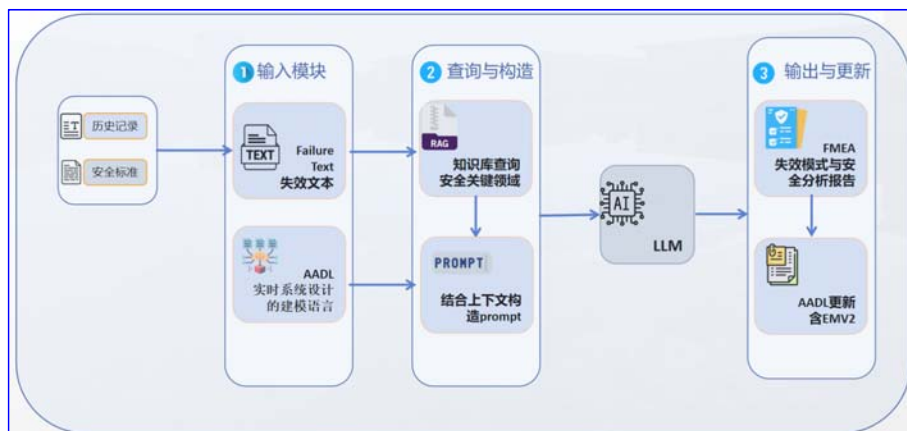


图 5.1 FMEAanalysis 模块设计

5.1.1 FMEAanalysis 工具设计

(1) 工具任务

借助大语言模型的分析与推理能力，FMEAanalysis 工具支持用户基于 AADL 模型生成 FMEA 报告，并自动更新模型中的 EMV2 附件信息。主要技术栈：

OSATE 平台支撑

工具构建于 OSATE 平台之上，通过插件机制接入，支持对 AADL 模型的建模、实例化和语义提取。

Java

工具以 Java 语言为核心进行插件开发，实现业务逻辑控制、模型解析、图形界面构建等功能。Java 代码负责驱动工具的文件读取、配置选择、模块调用及最终输出的整理与展示

LangChain

引入 LangChain 框架以构建与大语言模型交互的流水线。LangChain 支持上下文管理、多轮对话控制及外部工具调用，使得语言模型能够在连贯的语境中进行高质量推理与文本生成，保证 FMEA 报告的完整性与逻辑性。

LLM SDK

工具通过接入 LLM SDK 与底层大语言模型建立通信，支持自然语言指令的传递与返回结果的解析，实现模型的封装调用。

Embedding Model

利用嵌入模型将安全分析相关文档及标准转化为向量表示，并结合向量检索机制，支持语义相关信息的快速匹配与补充，为语言模型提供安全关键背景知识支撑，提升分析专业性。

大语言模型

底层依托大语言模型的强大生成与推理能力，结合基于“思维链”的提示工程策略，引导模型对失效路径、原因、影响和控制策略等进行系统性分析，自动生成结构化的 FMEA 报告内容。

(2) 工具界面设计

本工具是采用模块化设计，具体功能模块如下：

AADL 信息输入模块

功能：用于上传和 AADL 实例信息，提供文档预览功能，显示长传文件的具体内容。

失效文本输入模块

功能：接收与模型相关的失效描述信息，作为语言模型推理的重要输入。

配置区域模块

提供多项参数配置选项，包括：

知识库选择：指定用于语义检索的安全关键知识文档；

模型选择：选择所使用的大语言模型；

嵌入模型选择：配置用于向量化检索的 Embedding 模型

提示区域模块

功能：展示用于引导大语言模型推理的思维链提示词，用于引导模型按步骤完成 FMEA 分析。

FMEA 报告展示模块

功能：展示自动生成的 FMEA 报告内容

FMEAAalysis 的工具界面如图 5. 2。主要功能模块的代码统计如表错误!未找到引用源。。

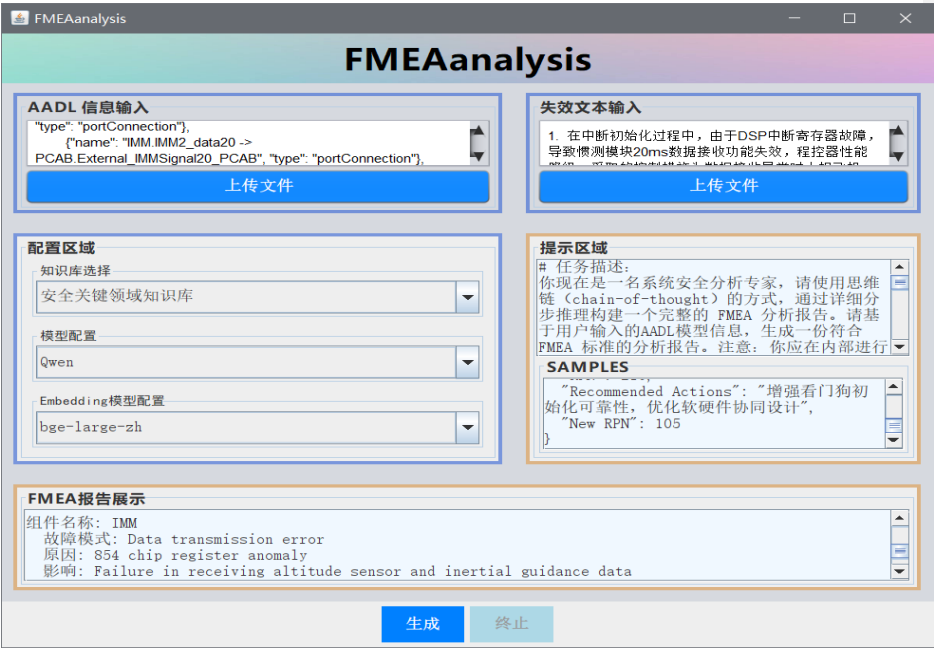


图 5. 2FMEAAalysis 工具界面展示

表 5. 1FMEAAalysis 主要功能模块的代码统计

模块	功能	Java 代码行数
菜单界面	用户操作接口	约 200
文件输入	读取文件并展示	约 200
提示模板	为用户推荐相关提示模板	约 200
基础配置	对大模型，知识库等进行选择和配置	约 300
报告展示	生成的 FMEA 报告展示	约 300

5.2 案例分析

弹射座椅控制系统属于典型的安全关键嵌入式系统，此系统是专门为让飞行员在紧急状况下可迅速且安全地离开飞机而设计的，该系统是由座椅本体、弹射装置、降落伞、数字电子序列器也就是控制系统以及相应的传感器和执行机构共同构成控制体系，系统会对飞行器的状态进行实时监测比如空速、高度、姿态等，并且依据预先设定的多种控制模式和算法，在契合启动条件的时候触发弹射操作。现代弹射座椅整合了十多种控制模式，可以依据环境以及飞行参数的变化动态地挑选最优策略，以此来兼顾性能和安全性，为保证系统在各种极端条件下可可靠运行，弹射座椅控制系统要经过严苛的测试和认证流程，涉及硬件冗余、故障注入测试、环境试验等，并且运用行业标准来进行合规性验证。

对于系统建模与分析，AADL 提供了图形化和文本化两种方式，在实际工程中大多以文本建模为主，文本建模便于精确表达安全关键属性与行为，而使用图形化为辅，用于评审与可视化展示。

进行弹射座椅控制系统 AADL 模型的建模过程如下：

（1）需求分析与功能分析

首先，根据工业界提供的系统需求文档对弹射座椅控制系统的总体需求进行全面归纳与提炼。其次，通过自上而下的功能分解方法，将飞行员逃生所需的关键功能拆分为若干可验证的子功能单元，并在功能分解图中明确各子功能之间的输入/输出关系及依赖约束。

（2）系统架构设计

基于需求分析，使用 AADL 语言构建系统的逻辑与物理架构模型。首先在逻辑视图中，通过 **System** 组件及其子组件描述软件功能单元及其内部交互，并采用端口/连接对组件间通信进行建模，以支持行为分析与可视化验证。随后，在物理视图中，将逻辑组件映射到执行平台实体，定义运行环境及部署拓扑，以便开展时序分析、资源评估与容错验证。整个架构遵循层次化组件复用原则，并在模型中标注调度协议、时序约束与容错策略，为后续性能验证与安全分析提供依据。

（3）接口与交互建模

在 AADL 接口建模中，通过数据端口、事件端口和事件数据端口精确定义组件间的同步与异步通信，使用 **connect** 语句关联端口并在属性中标注传输延迟与队列深度，结合文本与图形化视图，可快速生成通信矩阵并直观验证模块间交互一致性与时序约束。

如图 5.3 所示，弹射控制系统的核心组件包括 EEPROM 存储器、FWD 看门狗进程、PCAB 程序控制模块、PCC 程序控制模块、三组弹射启动开关（K1、K2、K3）、HDS 高度数据管理模块、ED 电爆管管理单元、Eth 总线以及 CPU 中央处理器等。图中箭头展示了数据流和控制信号的传输路径。程序控制模块，既 PCAB 和 PCC 作为系统中枢，负责与其他组件进行数据交互和信号传递，统筹管理各单元运行，确保系统高效、稳定地执行弹射任务。

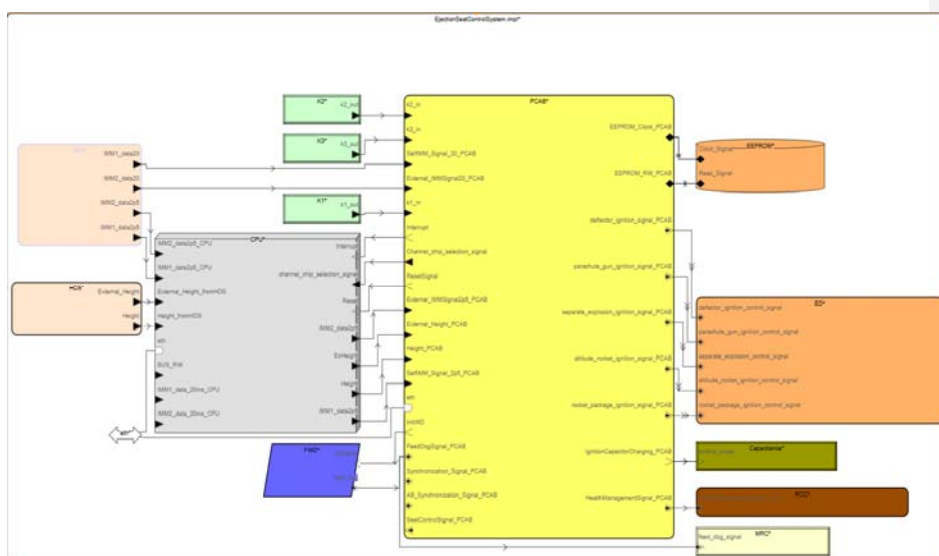


图 5.3 弹射控制座椅 AADL 模型图形化部分建模

弹射座椅控制系统 AADL 模型的文本建模示例如下：

```
package ES2::packages::AB_Refine
public
  with devices;
  with systems;
  with processes;

  system EjectionSeatControlSystem
end EjectionSeatControlSystem;

system implementation EjectionSeatControlSystem.impl
  subcomponents
    --三路弹射启动开关
    K1:device devices::K1;
    K2:device devices::K2;
    K3:device devices::K3;
    --分离, 射伞点火电容
    Capacitance: device devices::Capacitance;
    CPU:processor devices::CPU;
    --总线
    eth:bus devices::ethernet;
    --两路惯测模块系统
    IMM: system systems::InnertialMeasurementModuleHandling.impl;
    --高度数据管理系统
    HDS: system systems::HeightDataSystem.impl;
    --电爆管管理系统
    ED: system systems::ElectricDetonatorSystem.impl;
    --喂狗进程
    FWD: process processes::FeedWatchDog.impl;
    --存储器
    EEPROM: memory devices::EEPROM;
    --程控模块 AB
    PCAB: system systems::PCAB.impl;
    --程控模块 C
    PCC: system systems::PCC.impl;
    --监控复位电路
    MRC: device devices::MonitoringResetCircuit;

  Connections
end ElectricDetonatorSystem.impl;
```

图形化建模和文本建模描述的是同一系统架构，仅表达方式不同。图形化建模通过直观的界面展示系统架构，便于设计和理解，而文本建模则通过编写符合 AADL 语言规范的代码，精确定义系统结构和行为。两者在内容上等价，支持相互转换：设计人员可在图形

批注 [A11]: 代码字大一些

化工具中操作，自动生成文本代码。或直接编写 AADL 文本模型，由工具转化为图形化表示，确保设计一致性和灵活性。

5.2.1 基于 FMEAanalysis 的 FMEA 报告生成

为验证 FMEAanalysis 工具的效果，使用弹射座椅控制系统进行验证。首先，对于弹射座椅控制系统的 AADL 实例化信息进行压缩。

弹射座椅控制系统 AADL 模型实例化后的文件大小为 293.36KB，文件内容如下：

```
</componentInstance>
<componentInstance name="K1" category="device">
  <featureInstance name="k1_out" direction="out">
    <feature xsi:type="aadl2:DataPort"
href=" ../devices.aadl#0/@ownedPublicSection/@ownedClassifier.0/@ownedDataPort.0"/>
  </featureInstance>
  <subcomponent xsi:type="aadl2:DeviceSubcomponent"
href=" ../AB_Refine.aadl#0/@ownedPublicSection/@ownedClassifier.1/@ownedDeviceSubco
mponent.0"/>
    <index>0</index>
    <classifier xsi:type="aadl2:DeviceType" href=" ../devices.aadl#devices.K1"/>
  </subcomponent>
</componentInstance>
```

进行信息压缩后的文件大小为 44.81KB，文件内容如下：

```
{
  "componentName": "eth",
  "category": "bus",
  "parent": "EjectionSeatControlSystem_impl_Instance",
  "connections": [
    { "name": "eth -> PCAB.eth", "type": "accessConnection" },
    { "name": "eth -> CPU.eth", "type": "accessConnection" }
  ],
  "properties": [
    { "name": "BandWidthCapacity", "value": "500000.0" }
  ]
}
```

通过 AADL 信息压缩，成功对弹射座椅控制系统的 AADL 实例化进行信息提取。将提取到的信息结合上失效文本输入工具。成功得到 128 条对应的 FMEA 报告。根据 RPN 值分为不同风险级别：高风险、中风险以及低风险。

- 高风险失效模式：在 FMEA 分析中，弹射座椅控制系统有 2 种高风险失效模式，得分在 600 以上，分别是：中断初始化失败和系统主频初始化失败。
- 中风险失效模式：在 FMEA 分析中，弹射座椅控制系统有 14 种中风险失效模式，得分在 300 到 600 之间。
- 低风险失效模式：弹射座椅控制系统中有 112 种低风险失效模式，通常对系统的影响较小。

针对中高风险失效模式，采取的相应安全措施示例如下：

(1)中断初始化失败

采用冗余方案：为关键中断信号配置软硬件双重冗余设计，即使某一路中断初始化失败，备用路径仍可保障系统的正常运行。

(2)系统主频初始化失败

引入复位机制：设计合理的系统复位流程，确保在主频初始化失败时，系统能够自动重启并重新尝试初始化，避免系统停滞。

采取对应的安全措施后，高风险和中风险失效模式都转变为低风险失效模式，提高了系统可靠性，降低了事故风险。结果证明了 FMEAanalysis 工具在生成 FMEA 报告的有效性。

表 5. 2FMEA 报告示例

批注 [A12]: 三线表

Component Name	Mode	Cause	Effect	原始值				Recommended Actions	New RPN
				Severity	Occurrence	Detection	RPN		
ED1 (device)	点火执行机构失效	电爆管线路断路/点火药剂受潮	姿态火箭未点火	9	3	7	189	采用冗余桥丝设计，增加湿度传感器实时监测	95
IMM1 (device)	惯性数据漂移超限	MEMS 传感器温漂/校准失效	姿态解算误差累积	7	5	6	210	植入温度-误差补偿矩阵，设计自校准激励机构	120
EEPROM (memory)	配置参数存储异常	写操作电压不稳/存储单元老化	系统初始化失败	8	4	8	256	采用铁电存储器替代方案，增加存储分区健康状态监测	134

5.2.2 基于 FMEA 报告的 EMV2 附件扩展

本节基于由 FMEAnalysis 工具对弹射座椅控制系统进行分析后生成的 FME 报告，对系统的 AADL 模型进行 EMV2 附件扩展。通过将 FMEA 报告与上文映射规则一并输入大语言模型，生成扩展后的 EMV2 错误类型库及属性集，如错误!未找到引用源。。

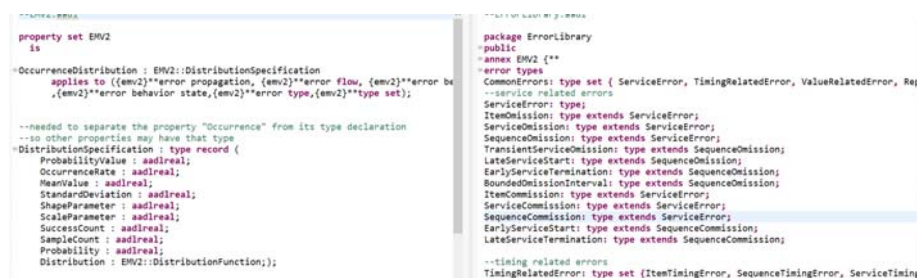


图 5.4 扩展的属性集以及错误类型库

随后，将已建模的 AADL 组件及其对应的 FMEA 报告再次输入大语言模型，自动生成匹配的 EMV2 附件内容，实现对每个组件的故障建模扩展，如错误!未找到引用源。。

```

with EMV2;
device device_1
features
  out_feature_1: out feature;
annex emv2 {**
  use types test1;

  error propagations
    out_feature_1: out propagation {error_type_1};
  end propagations;

  component error behavior
    events
      events_1: error event;
      events_2: error event;
    propagations
      all -[events_1 and events_2]-> out_feature_1 {error_type_1};
    end component;

  properties
    EMV2::OccurrenceDistribution => [
      ProbabilityValue => 0.5;] applies to events_1;
    EMV2::OccurrenceDistribution => [
      ProbabilityValue => 0.2;] applies to events_2;
  **};
end device_1;

```

图 5.5 扩展的 AADL EMV2 附件样例

5.3 讨论

本节从工具的应用效果以及方法的局限性两个方面，对 FMEAnalysis 工具的性能和局限性进行了深入讨论。

（1）工具效果

FMEAanalysis 工具实现了从系统建模到 FMEA 报告生成的端到端自动化流程。在弹射座椅控制系统的验证过程中，该工具成功从 290KB 以上的实例化模型中提取出关键结构与行为信息，并在此基础上结合失效案例生成包含 128 条失效模式的 FMEA 报告，同时给出了对失效模式相应的安全建议与设计改进。此外，成功自动扩展模型的 EMV2 附件。

（2）方法的局限性

尽管本研究展示了基于大语言模型的 FMEA 分析方法的可行性，但仍存在一些局限性。首先工具对上下文的依赖性强，生成的 FMEA 报告质量依赖于 AADL 模型信息以及相关失效案例的输入。其次部分结果仍需人工校验，尽管大语言模型可自动生成 FMEA 报告以及结构正确的 EMV2 片段，但在处理复杂逻辑，如多状态交叉故障传播、非线性依赖路径时，仍需专家进行人工干预与结果验证。此外，对于 RGA 系统而言，安全关键领域的知识未实现动态更新。

5.4 本章小节

本章介绍了 FMEAanalysis 工具的设计与实现过程，并以弹射座椅控制系统为案例验证其有效性。该工具基于 OSATE 平台，结合 Java 插件开发、LangChain 交互与大语言模型推理，实现了 AADL 模型到 FMEA 报告的自动生成。并进行了弹射座椅控制系统案例分析。在案例中成功生成 128 条失效模式报告，并提出对应的有效改进措施。最后，通过 FMEA 结果自动扩展 EMV2 附件，增强了模型的故障建模能力。

第六章 总结与展望

6.1 总结

本文针对安全关键软件失效模式与影响分析中人工分析效率低、覆盖不全、主观性强等问题，提出了一种基于大语言模型的自动化 FMEA 生成方法。通过结合 AADL 模型驱动、检索增强生成技术及思维链提示框架，实现了从系统模型到安全分析的端到端流程。主要工作如下：

(1) 对于人工进行失效模式与影响分析耗时耗力的问题，提出基于大语言模型的失效模式与影响分析方法。首先，进行 AADL 信息压缩用于输入大语言模型。其次通过检索增强生成，为 LLM 提供安全关键领域提供专业领域知识，然后，通过 Prompt 思维链方法，并结合 AADL 组件拓扑关系引导 LLM 生成标准化、可追溯的 FMEA 分析条目，最后根据生成的 FMEA 报告扩展 AADL 模型。

(2) 对基于大语言模型的失效模式与影响分析方法进行实验分析。进行了三个实验，分别验证了信息压缩的效率、RAG 获取安全关键领域知识的效果以及基于思维链的 FMEA 提示框架相较于直接提示的优势。

(3) 设计并开发原型工具 FMEAanalysis。该工具基于 AADL 建模环境 OSATE,用于失效模式及影响分析。为证明该工具的有效性，进行了工业界弹射座椅控制系统的案例分析。实现了从原始 AADL 模型到 FMEA 报告的端到端生成。验证了本文工具方法的有效性。

6.2 展望

在本文现有工作的基础上，考虑到方法的局限性，在未来需要进一步解决的问题有：

(1) 知识库增强：为了能让安全关键知识库的实用性以及时效性得到提升，进而引入动态更新机制，此机制可支持对最新行业标准与案例进行实时检索，将用户行为分析与反馈机制相结合，系统可识别出高频查询以及潜在知识盲区，自动触发内容优化流程，以此保证知识库可维持高效、实用且可靠的状态。

(2) 可信性验证拓展：运用形式化方法来保障 FMEA 报告有可信性，借助模型检查工具去验证失效传播路径的逻辑完备状况，以此保证风险量化的结果契合专家经验分布，在报告当中嵌入元数据标签，构建起从生成条目直至原始模型的双向追溯链路，提高结果的可审计性，减少人工复核所需的成本。

参考文献

- [1] SINGH P, SINGH L K. Reliability and Safety Engineering for Safety Critical Systems: An Interview Study With Industry Practitioners [J]. IEEE Transactions on Reliability, 2021, 70(2): 643-653.
- [2] SAE INTERNATIONAL. Guidelines for Development of Civil Aircraft and Systems: SAE Standard ARP4754A [S]. Warrendale: SAE International, 2010.
- [3] SAE AIRCRAFT SYSTEMS DEVELOPMENT COMMITTEE. Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment [S]. Warrendale: SAE International, 1996.
- [4] FRIEDENTHAL S, MOORE A, STEINER R. A practical guide to SysML: the systems modeling language [M]. Morgan Kaufmann, 2014.
- [5] FEILER P H, GLUCH D P. Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language [M]. Addison-Wesley Professional, 2012.
- [6] PROSVIRNOVA T, BATTEAUX M, BRAMERET P A, et al. The AltaRica 3.0 Project for Model-Based Safety Assessment [J]. IFAC Proceedings Volumes, 2013, 46(22): 127-132.
- [7] Stewart D, Liu J, Cofer D, et al. AADL-Based Safety Analysis Using Formal Methods Applied to Aircraft Digital Systems[C]. 10th Edition European Congress Embedded Real Time Systems, 2020.
- [8] Sun M, Gautham S, Ge Q, Elks C, Fleming C. Defining and Characterizing Model-based Safety Assessment: A Review[J]. Journal of Safety Science, 2024, 10(1): 1-27.
- [9] Sun M, Fleming C H, Milich M. Defining and Reasoning about Model-based Safety Analysis: A Review[R]. NASA Langley Research Center, Hampton, Virginia 23681-2199, 2021: 1-49. NASA/CR-20205009755.
- [10] Hu X, Liu J, Dou H, Chen H, Zhang Y. Automatic Generation of Component Fault Trees from AADL Models for Design Failure Modes and Effects Analysis[J]. In: 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS), 2023: 550-559. DOI: 10.1109/QRS60937.2023.00060.

-
- [11] El Hassani I, Masrour T, Kourouma N, Motte D, Tavakoli J. Integrating large language models for improved failure mode and effects analysis (FMEA): a framework and case study[J]. Design, 2024.
 - [12] Xia Y, Jazdi N, Weyrich M. Enhance FMEA with Large Language Models for Assisted Risk Management in Technical Processes and Products[C]//2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA). IEEE, 2024.
 - [13] Razouk H, Liu X L, Kern R. Improving FMEA comprehensibility via common-sense knowledge graph completion techniques[J]. IEEE Access, 2023, 11: 127974-127986.
 - [14] Bahr L, Wehner C, Wewerka J, Bittencourt J, Schmid U, Daub R. Knowledge Graph Enhanced Retrieval-Augmented Generation for Failure Mode and Effects Analysis[J]. arXiv preprint arXiv:2406.18114, 2024.
 - [15] Bubeck S, Chandrasekaran V, Eldan R, et al. Sparks of Artificial General Intelligence: Early experiments with GPT-4R. arXiv preprint arXiv:2303.12712, 2023
 - [16] 张钦彤, 王昱超, 王鹤羲, 王俊鑫, & 陈海. (2024). 大语言模型微调技术的研究综述. Journal of Computer Engineering & Applications, 60(17).
 - [17] Hu E, Shen Y, Wallis P, et al. LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS[J]. arXiv preprint arXiv:2106.09685, 2021.
 - [18] Elad Ben-Zaken, Shauli Ravfogel, Yoav Goldberg. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models[J]. arXiv preprint arXiv:2106.10199, 2022.
 - [19] LI X L, LIANG P. Prefix-Tuning: Optimizing Continuous Prompts for Generation [A]. In: Proceedings of the 59th Annual Meeting of the ACL and the 11th IJCNLP, Volume 1: Long Papers [C]. Online: Association for Computational Linguistics, 2021: 4582-4597.
 - [20] LESTER B, AL-RFOU R, CONSTANT N. The Power of Scale for Parameter-Efficient Prompt Tuning [C]. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP). Punta Cana: Association for Computational Linguistics, 2021: 3045-3059.
 - [21] HOULSBY N, GIURGIU A, JASTRZEBSKI S, et al. Parameter-Efficient Transfer Learning for NLP [A]. In: Proceedings of the 36th International Conference on Machine Learning [C]. Long Beach, California: PMLR, 2019: 2719-2728.

- [22] ARSLAN M, GHANEM H, MUNAWAR S, et al. A Survey on RAG with LLMs [A]. In: Proceedings of the 28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024) [C]. Dijon, France: ELSEVIER B.V., 2024: 3781-3790.
- [23] RAFAEL C A, BURNS M. Zero-Shot Text Classification with Pretrained Language Models [C]. In: Proceedings of the 2021 Conference of the North American Chapter of the ACL (NAACL). Association for Computational Linguistics, 2021.
- [24] BROWN T, MANN B, RYDER N, et al. Language Models Are Few-Shot Learners [J]. Advances in Neural Information Processing Systems (NeurIPS), 2020, 33:1877-1901.
- [25] WEI J, BOSMA M, XU J, et al. Inverse Prompting: Querying Language Models for Their Reasoning [C]. In: Proceedings of the 2022 Conference on Neural Information Processing Systems (NeurIPS). Curran Associates, Inc., 2022.
- [26] FINN C, ABBEEL P, LEVINE S. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks [C]. In: Proceedings of the 2017 Conference on Neural Information Processing Systems (NeurIPS). Curran Associates, Inc., 2017.
- [27] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [EB/OL]. arXiv:2201.11903, 2023. <https://arxiv.org/abs/2201.11903>.
- [28] 杨志斌, 皮磊, 胡凯, et al. 复杂嵌入式实时系统体系结构设计与分析语言:AADL [J]. 软件学报, 2010, 21(05): 899-915.
- [29] DELANGE J, FEILER P. Architecture fault modeling with the AADL error-model annex; proceedings of the 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications, F, 2014 [C]. IEEE
- [30] Feiler, P. (2004, October). Open source AADL tool environment (OSATE). In AADL Workshop, paris(pp. 1-40).
- [31] McDermott R E, Mikulak R J, Beauregard M R. The Basics of FMEA [M]. 2nd ed. CRC Press, 2008.

批注 [A13]: 年份, 页码格式不对